

Research paper

Asynchronous multithreading reinforcement learning with attention-based significance measurement for collision-free robot navigation

Chao Sun^a, Jiang Wang^a, Xing Wu^{b,*}, Chaoxu Mu^{a,c}, Changyin Sun^{a,d,**}^a School of Artificial Intelligence, Anhui University, Hefei, 230601, Anhui, China^b College of Mechanical and Electrical Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, 210016, China^c School of Electrical Automation and Information Engineering, Tianjin University, Tianjin, 300192, China^d School of Automation, Southeast University, Nanjing, 210096, China

ARTICLE INFO

Keywords:

Robotic vehicle

Reinforcement learning

Collision-free navigation

Significance measurement

Local attention

ABSTRACT

Collision avoidance is one crucial technique to achieve safe and efficient robotic vehicle navigation in unknown environments. However, moving obstacles with unpredictability in dynamic scenarios, usually increase the difficulty and complexity in collision avoidance of robotic vehicles. To enhance the stability of collision avoidance and boost its adaptability to uncertain dynamic scenes, a new attention-based significance measurement actor-critic (ASMAC) architecture is proposed. It is an end-to-end robot navigation model that uses imperfect local observation to directly plan precise collision-free motion commands. Firstly, a significance-measured rollout replaybuffer (SMRR) is presented to categorize the experiences into different pools. It can prevent any overfitting or bias that may result from repeatedly sampling experience of a certain type during policy learning. Then, we enhance the traditional actor-critic network by integrating a multi-head local attention module to extract the local information at entity level. This way, the collision avoidance system can focus on key environmental features to compute more lightweight and respond more swiftly to dynamic changes in environment. Besides, a multi-step lookahead prediction (MLP) reward function is designed in the ASMAC-based reinforcement learning (RL) framework to prevent the generation of unnatural, intrusive, and short-sighted motion decisions. Finally, the asynchronous multithreading (AM) mechanism and proximal policy optimization (PPO) algorithm are extended to ASMAC model to offload the expensive online computation to an offline training process, enhancing the exploration efficiency in navigation policy learning of robotic vehicles. Extensive simulation and real-world physical experiments show that our method can generate time-efficient and collision-free guide paths in complex dynamic scenes, to successfully dodge collisions while moving towards the goal.

1. Introduction

Automated guided vehicle (AGV) is one typical industrial robotic vehicle utilized for repeated transportation tasks in factories for more than half of a century (Wu et al., 2020; Sun et al., 2022). With the wide application of AGV in disaster rescue, restaurant delivery, and household service, the collision avoidance methods have become increasingly important during AGV navigation. It plays a vital role in enabling AGVs to operate autonomously and safely in the environments. While there has been significant progress over the past decade (Sheng et al., 2025; Zhang et al., 2024), safely and fully autonomous navigation is still challenging, especially in dynamic and uncertain workspaces cohabited by other moving objects, e.g., vehicles, pedestrians or other

robots. The challenge usually arises due to the lack of knowledge of the navigation system about the intentions and strategies of other dynamic objects. To this end, the system endows AGV with the capabilities of collision risk assessment and robot-obstacle interaction, which are of great significance for AGVs to autonomously plan a collision-free path in complex dynamic scenes. Recently, some collision avoidance methods for AGV have been developed to avoid moving objects in dynamic scenes. According to task hierarchy, current collision-free navigation approaches can be divided into global and local approaches. The former is based on known obstacles map to generate continuous paths to avoid collisions (Xu et al., 2025; Shen et al., 2014). This process involves two crucial points. An environment map with obstacles

* Corresponding author.

** Corresponding author at: School of Artificial Intelligence, Anhui University, Hefei, 230601, Anhui, China.

E-mail addresses: 24064@ahu.edu.cn (C. Sun), wangjiang@stu.ahu.edu.cn (J. Wang), wustar5353@nuaa.edu.cn (X. Wu), cxmu@tju.edu.cn (C. Mu), cysun@ahu.edu.cn (C. Sun).<https://doi.org/10.1016/j.engappai.2026.113779>

Received 14 July 2025; Received in revised form 1 January 2026; Accepted 6 January 2026

Available online 8 January 2026

0952-1976/© 2026 Published by Elsevier Ltd.

Table 1

Summary of related works of collision-free navigation systems in complex dynamic scenes.

Reference	Proposed method	Year	Collision-free strategy	Limitation
Malone et al. (2017)	SRS-APF	2017	Event prediction	Geometric rule strategy: struggles to handle highly dynamic scenarios existing numerous moving obstacles.
Lin et al. (2025)	TD-APF	2025	Space planning	
Seo et al. (2020)	FB-RA	2020	HJ formulation	
Chen et al. (2018)	BF-RA	2018	Barrier function	
Poonganam et al. (2020)	NPU-NMPC	2020	Probabilistic velocity	Optimization searching strategy: easily plunges into local minima.
Lee et al. (2020)	FDE-DWA	2020	Finite distribution	
Missura and Bennewitz (2019)	PCA-DWA	2019	Predictive collision	
Sheng et al. (2025)	M-DQN	2025	Improved m-dqn	Reinforcement learning strategy: depends heavily on the quality of labeled data or demonstrations.
Sun et al. (2024)	AVCAC	2024	Value classification	
Zhou et al. (2024)	AL-DSAC	2024	Representation learning	
Dong et al. (2024)	TSG-AC	2024	Graph attention	
Soares and Campos (2025)	SL-MDE	2025	Self-supervised learning	

is first built offline/online via simultaneous localization and mapping (SLAM) technique, followed by planning collision-free trajectory with respect to the map for safe navigation. Yet, in a scene full of moving obstacles, SLAM may fail frequently because of noises or occlusions, resulting in the inability to generate reliable maps (Cheng et al., 2022).

To address the above issues, the local collision avoidance navigation techniques have been proposed recently. It is one kind of mapless navigation solution, in which AGVs typically utilize the real-time imperfect local sensing rather than prior knowledge to generate temporary avoidance paths for safe and efficient navigation (Guo et al., 2022; Fan et al., 2020). In this context, this group of techniques is more applicable to practical robotic application for collision-free navigation. Hence, this group of collision avoidance method is focused on this paper. Table 1 summarizes the notable examples of local collision-free navigation systems that utilize mapless technique for accurate and safe collision avoidance in complex dynamic scenes.

According to the technical route, conventional mapless navigation methods can be divided into two strategies. One is the geometric rule strategy. It simplifies the obstacles as convex bodies, followed by computing the lower and upper boundaries of collision-free navigation path. The typical methods include artificial potential field (APF) (Huang et al., 2019; Lin et al., 2025) and reachability analysis (RA) (Seo et al., 2019; Pham and Pham, 2018). APF method and its variants rely primarily on the repulsive force engendered when approaching the obstacles to drive the AGV to dodge collisions. While APF-based solutions are efficient and easy to implement, many of them neglect future behavior of dynamic obstacles. It may cause troubles in AGV collision avoidance when the dynamic changes of moving obstacles are large. To alleviate this issue, Malone et al. (2017) enhance the APF with a Markov-based future event prediction module to assess collision threat, but leading to a high computational cost.

The RA is a reachable set-based obstacle avoidance solution, which is calculated by polar technique and Hamilton–Jacobi (HJ) formulation. It is a kind of target set that the robot in a certain initial state can safely reach through control in the presence of any disturbance. In Seo et al. (2020), a robust collision-free navigation approach is investigated by constructing the forward and backward reachable sets using HJ formulation. Chen et al. (2018) compute the reachable set using polar algorithm to achieve collision avoidance, in which the obstacle avoidance set is defined as the complement of controlled invariant. While these solutions adopt Markov process approximation to decrease the runtime of probability computation for reachable sets, they struggle to handle highly dynamic scenarios existing numerous moving obstacles.

The second is path optimization searching technique. Many methods (Xiao et al., 2016; Poonganam et al., 2020) apply nonlinear model prediction control (NMPC) to avoid collisions. The NMPC can combine various constraints to gain optimal control and real-time operation in the process of collision avoidance. Yet, as the number of obstacles increases, the vector dimension of constraint increases rapidly, which can decrease the performance of NMPC dramatically. In Lee et al. (2020), the dynamic window method (DWM) is used to gain the samples from

velocity space of AGVs, followed by predicting its possible collision-free motion trajectory via an objective function. Missura and Bennewitz (2019) improve the DWM to model moving object as a polygonal object with velocity, and compute future collisions. Nevertheless, these studies easily plunge into local minima, since DWM solely considers the AGV's goal heading and lacks free-space connection information.

As artificial intelligence (AI) advances, many mapless navigation methods using reinforcement learning (RL) have gained attention. They map raw local observations to collision-free steering command directly (Sui et al., 2020; Kiumarsi et al., 2017; Everett et al., 2021). RL is one of machine learning algorithms, which can learn and optimize the navigation policy in a “trial and error” fashion, through interacting with the environments. Unlike conventional methods, it has a strong adaptability to model-free feature and complex system, which provides a new method to the collision-free navigation problem. In Lin et al. (2023), a transformer-based dual-channel self-attention term is proposed to endow AGVs with the ability to assess collision threats, thereby making collision avoidance decisions. The work (Sun et al., 2024) introduces one attention-based value classification actor–critic architecture into RL objective to promote more informed navigation decisions. Wu et al. (2021) utilize grid-map image as state input to develop dynamic collision situations. However, the aforementioned studies only perform well in relatively simple and structured test environments. Hence, these RL-based navigation models are hard to generalize to complex and highly dynamic scenes.

To address this difficulty, a discrete soft actor–critic method that combines data replay and attention encoding is implemented to enhance the performance of RL (Zhou et al., 2024). In Song et al. (2024), Dong et al. (2024), researchers utilize the global information to guide the update of decentralized actor network in light of assuming that each robot is fully aware of the environment. This setting is suitable for handling small scene tasks. In real application, the field of view (FOV) of each AGV is limited by the sensing range of dominant sensor. Therefore, some researchers select raw sensor data (e.g., Lidar Xia et al., 2023 or Camera Soares and Campos, 2025) as input to design end-to-end RL patterns to realize robot collision avoidance in real scenes. The sensor-level techniques can deal with the variable observation dimension well, but introduce interpretability and large-size input problems. In addition, Godoy et al. (2016) predict dynamic collision in advance by using Bayesian inference mechanism and calculate collision-free actions utilizing optimal reciprocal collision avoidance framework. Long et al. (2018) employ multi-scene multi-stage training paradigm to estimate the state of moving obstacle for learning optimal policy. In Qureshi et al. (2021), scholar uses supervised learning to teach AGV's planning collision avoidance policy. Whereas the effectiveness in avoiding collision has been demonstrated through simulation test, the actual performance of this way depends heavily on the quality of labeled data or presentation. To sum up, current collision-free navigation methods still remain many limitations, when operating in the imperfect sensing, large-scale search, and dynamic uncertain workspaces.

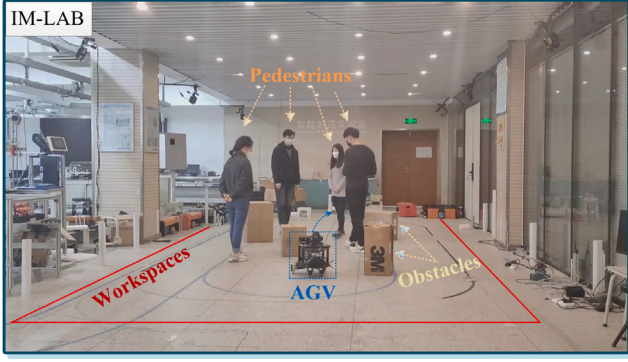


Fig. 1. AGV navigation with limited sensing in complex dynamic environments.

To address the problems mentioned above, a novel attention-based significance measurement actor-critic (ASMAC) collision avoidance method is presented in this paper. It is a model-free end-to-end robotic vehicle navigation method that can enable AGVs to navigate safely and efficiently in various dynamic scenes (see Fig. 1). Our major contributions are as follows:

- Unlike most existing synchronous RL-based obstacle avoidance solutions (Fan et al., 2020; Long et al., 2018; Liu et al., 2023), a novel RL approach that extends asynchronous multithreading mechanism and proximal policy optimization to the actor-critic paradigm is developed to learn optimal collision avoidance policy. This way can take full advantage of representation capabilities of local observation information to offload the expensive online computation to an offline training procedure, thus ensuring safe and efficient navigation of AGVs in dynamic uncertain scenarios.
- Different from using standard uniform replaybuffer in Everett et al. (2021), Dong et al. (2024), a significance-measured rollout replaybuffer is developed to enhance the sample utilization and mitigate the variance. Also, unlike existing interaction attention strategies in Dong et al. (2024), Liu et al. (2023), a new state encoder termed local attention module (LAM), is introduced to extract and encode entity-level local information. It can provide a more lightweight representation and capture collision-relevant environmental feature. Through the tight cooperation between rollout replaybuffer and state encoder, the navigation model can boost its perception capability and learning effectiveness, thus making safe and effective collision avoidance decisions rapidly.
- Unlike existing collision prediction tricks (Lin et al., 2025; Misra and Bennewitz, 2019; Liu et al., 2023; He et al., 2022) deployed for collision avoidance, we innovatively propose a multi-step lookahead prediction (MLP) reward function in RL system during the training phase to anticipate potential collision threat in advance. This prevents the navigation system from generating shortsighted, intrusive, and unnatural motion policies.

The remainder of this paper is organized as follows. The second section formulates the collision avoidance issue and introduces the AGV's control model. A detailed discussion of our method is presented in the third section. The experiment procedure and results are shown in the fourth section. Finally, conclusions and recommendations for future research are given in the last section.

2. Problem formulation

From the perspective of RL, the collision avoidance navigation task can be modeled as a partially observable Markov decision problem

(POMDP) (Oliehoek et al., 2016). Formally, the POMDP is described as a 6-tuple $(\mathcal{A}, \mathcal{S}, \mathcal{R}, \mathcal{T}, \Omega, \mathcal{O})$, where $\mathcal{A}$ is action space ($\mathbf{a} \in \mathcal{A}$), $\mathcal{S}$ is global state of environment ($s \in \mathcal{S}$), $\mathcal{R}$ is the reward function ($r \in \mathcal{R}$), $\mathcal{T}$ is the transition probability between states $\mathcal{T}(s' | s, \mathbf{a})$, Ω is observation space ($\mathbf{x} \in \Omega$), and $\mathcal{O}$ is observation distribution given the state ($\mathbf{x} \sim \mathcal{O}(s)$).

In the process of collision avoidance task, the AGV leverages the navigation policy learned from its own limited local observations $\mathbf{x}_t$ to execute the action $\mathbf{a}_t$ at each timestep t . Then, the environment reward feedback is gained, and while the navigation system reaches the next state. The goal of this process is to drive AGVs to approach target autonomously and safely, which corresponds to learning an optimal collision-free policy that maximizes the expected return E , that is:

$$E = \sum_{k=1}^{\infty} \lambda^{k-1} r_{t+k} \quad (1)$$

where $\lambda \in [0, 1]$ is the discount factor that can balance the long-term gain and short-term reward for the AGV.

It is noteworthy that the AGV tends to decelerate or accelerate abruptly during collision avoidance in real applications, since there is a sudden change between the expected velocity command $\mathbf{a}_t$ output by the navigation policy π_{θ} and the current velocity $\mathbf{v}_t$. This can cause the AGV to generate an unsmooth travel trajectory. Hence, a commonly-used proportion integration differentiation (PID) control method is embedded into our navigation system as the underlying action control strategy. It enables the AGV to take advantage of the stability of classical control algorithm for final executable, smooth, and continuous trajectory traveling. In addition, this PID controller is the operations on the denormalized physical velocity after model training, which solely smooths transitions between explored actions, not RL's action space itself. Hence, using PID controller does not limit RL policy's ability to explore or learn fine-grained actions. On the contrary, it focuses exploration on physically meaningful action spaces. It can be formulated as:

$$\mathbf{a}_t^{\text{real}} = \mathbf{v}_t + \mathbf{u}_t; \quad \mathbf{u}_t = K_p \mathbf{e}_t + K_i \int_{t_s=0}^t \mathbf{e}_t dt + K_d \frac{d\mathbf{e}_t}{dt}; \quad \mathbf{e}_t = \mathbf{a}_t - \mathbf{v}_t. \quad (2)$$

where t_s is the starting time, i.e., $t_s = 0$. K_p , K_i , and K_d are the coefficients of proportion, integration and differentiation, respectively. In detail, the PID gains K_p , K_i , and K_d are empirically set to 0.2, 0.1, and 0.02, respectively, to match AGV's physical constraints for ensuring PID output stays within hardware-feasible limits. In this way, the stability of AGV's motion control can be prioritized to avoid introducing additional variability that could disrupt the RL policy's learning process. $\mathbf{a}_t^{\text{real}}$ is the real velocity after being processed by the PID controller, representing the ideal continuous velocity the AGV should follow. However, in the real-world, AGVs operate at discrete timestep, so the continuous velocity must be converted to a discrete-time signal for hardware execution. To this end, we exploit the Eq. (3) to build a discrete-time model to update the AGV's self-state at each timestep. It directly aligns with AGV's practical operation, i.e., sensor data is sampled at discrete intervals, and RL navigation model generates corresponding actions at the same timesteps. Specifically, AGV's self-states (position and heading angle) at the next timestep are updated by:

$$\begin{bmatrix} p_{t+1}^x \\ p_{t+1}^y \\ \beta_{t+1} \end{bmatrix} = \begin{bmatrix} p_t^x \\ p_t^y \\ \beta_t \end{bmatrix} + \begin{bmatrix} \cos \beta_t \cdot v_t^l \\ \sin \beta_t \cdot v_t^l \\ v_t^w \end{bmatrix} \cdot \Delta t \quad (3)$$

where β_t is the AGV's heading angle. p_t^x and p_t^y are the horizontal and vertical components of the AGV's position $\mathbf{p}_t$, respectively. v_t^l and v_t^w are the translational and rotational velocities respectively, i.e., $\mathbf{a}_t = [v_t^l, v_t^w]$.

3. Methodology

In view of real-time, safe and efficient navigation requirements under complex dynamic scenes, a new attention-based significance measurement actor-critic (ASMAC) collision avoidance system is built

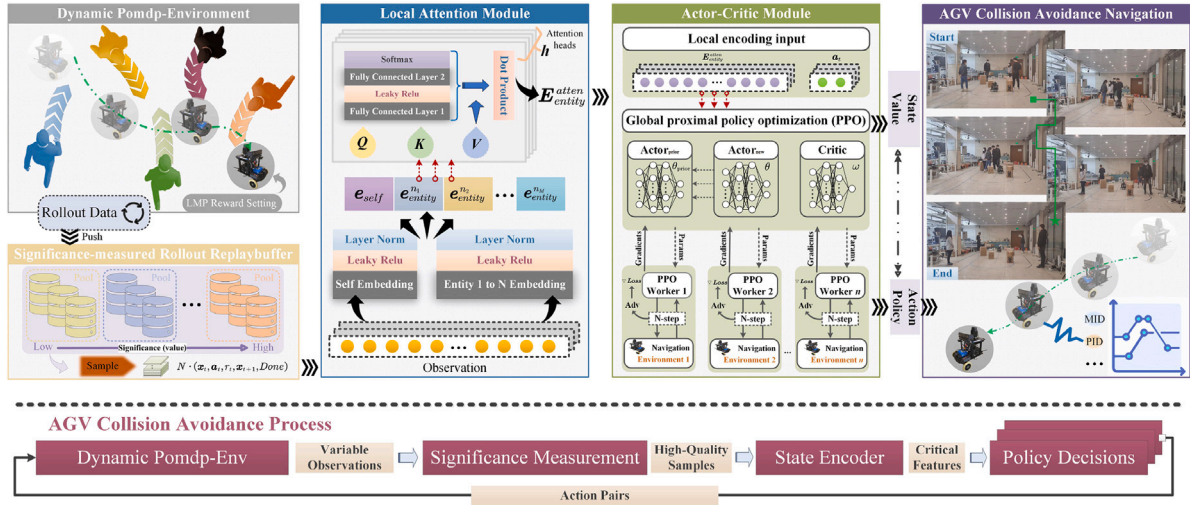


Fig. 2. The overall architecture of the attention-based significance measurement actor-critic (ASMAC) collision avoidance method.

to enable AGVs to make effective collision-free navigation decision. The overview architecture of our ASMAC method is shown in Fig. 2.

First, the AGV with imperfect perception navigates to its goal in a complex environment full of uncertain moving obstacles (e.g., pedestrians). In the process of navigation, AGV interacts with the environments to summarize the experience information for generating rollout data and transferring them into the significance-measured rollout replaybuffer (SMRR). Through the SMRR mechanism, these rollout data is partitioned and stored in different pools with fixed timestep sequence. This trick is much more practical and increase sample utilization effectively. Then, we design a local attention-based state encoder to extract and encode the critical environmental feature at the entity-level. It can measure the relative importance of the surrounding (static and dynamic) obstacles to assist the AGV make better navigation decisions. Furthermore, the local encodings are sent into the ASMAC model, which make collision-free navigation decision by introducing PPO. Meanwhile, the AM mechanism is adopted for policy learning. Through extending PPO and AM to the ASMAC, the performance of robot collision avoidance is effectively improved.

3.1. ASMAC configuration

Formally, the POMDP described as a 6-tuple can be solved using the RL framework during collision-free navigation. Below, we discuss ASMAC's configuration details, including observation space, action space, and reward function.

3.1.1. Observations

The information contained in observation space of an AGV includes preset prior data and measurement data from onboard sensors. In such case, AGVs cannot acquire the intents and states of other static and moving obstacles directly, but learns to make predictions and inferences from limited local observations, and make its own near-optimal or optimal collision-free action decision. This local observation way is in accord with the real-world situation compare to prior methods (Long et al., 2018; Liu et al., 2023) using a fully observable situation, since it is difficult for physical AGVs to accurately perceive the position and velocity of nearby moving obstacles. The specific observation setting of the AGV at timestep t is as follows:

$$\mathbf{x}_t = [p_t, v_t, p_t^g, \beta_t, \mathbf{x}_{\text{other}}] \quad (4)$$

where p_t and v_t are the global position and the velocity for the AGV, respectively. p_t^g denotes the global position of target point. β_t is the AGV's heading angle. $\mathbf{x}_{\text{other}}$ is an imperfect perception vector that integrates

the position of (static and dynamic) obstacles observed in FOV of AGVs at timestep t . It is noteworthy that $\mathbf{x}_t$ is a practical implementation of the POMDP's observation space Ω , listing the specific sensor-derived features (a subset of the POMDP) the AGV uses to make decisions.

3.1.2. Actions

Since the AGV in our environment utilizes the differential-driving wheels, a natural choice of action space for the AGV is a translational and rotational velocity, i.e., $\mathbf{a}_t = [v_t^l, v_t^w]$. In this work, the action space is continuously represented and is limited in a range of $[v_{\min}, v_{\max}]$. Considering the real robot's kinematic and real-world application, we scale the velocity range to $[-1, 1]$ during the training phase. This is a standard RL practice to stabilize policy optimization. Moreover, we assign goals to AGVs at each time step dynamically, which can allow the AGV to generate more flexible decisions. Also, it is noteworthy that this scaled output is not final velocity sent to AGV's hardware, but rather one reverse scaling step is applied after training to map the normalized $[-1, 1]$ values back to AGV's physical velocity ranges. Specifically, the reverse scaling step is formulated as: $v_{\text{physical}} = v_{\min, \text{native}} + (v_{\text{scaled}} + 1) / 2 \times (v_{\max, \text{native}} - v_{\min, \text{native}})$. Where the $[v_{\min, \text{native}}, v_{\max, \text{native}}]$ is the AGV's native output range, $v_{\text{scaled}} \in [-1, 1]$ is the scaled output, v_{physical} is the reverse scaling output, i.e., de-normalized velocity. Through this reverse scaling trick, it is possible to ensure that the actor-critic's output aligns with the AGV's hardware capabilities, eliminating any distortion of movements caused by training-specific normalization. In conclusion, scaling the actor-critic module's translational/rotational velocity outputs to $[-1, 1]$ is a training-specific optimization that does not distort the AGV's actual movements.

3.1.3. Rewards

The reward design directly determines whether the AGV can be guided effectively, so it plays a crucial role in the AGV to fulfill the task requirements. To make the AGV gain reward continuously, and avoid shortsighted and recklessness behaviors, we split the reward setting of the AGV into a MLP reward setting and a basic configuration. It is noteworthy that we assume that AGV has integrated object detection module that can accurately identify the interaction obstacles. This module is the foundation for distinguishing static and dynamic obstacles. It classifies obstacles in the AGV's field of view into two categories based on their motion state, i.e., static obstacles with no measurable motion across consecutive timestep and dynamic obstacles with detectable translational or rotational velocity.

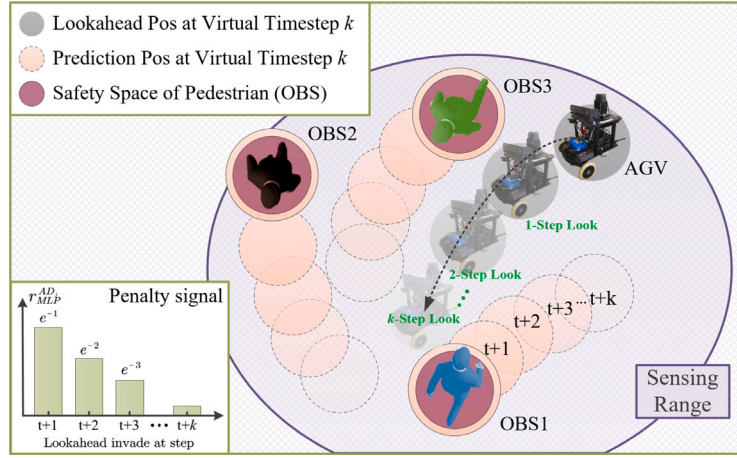


Fig. 3. AGV in its FOV at timestep t makes a multi-step trajectory prediction for moving obstacles (OBS), and executes multi-step uniform trajectory evolution on the basis of its current velocity. At each look-ahead step, AGV virtually invades the safety zone of moving obstacles or collides with it, while a specific penalty signal is produced. The magnitude of this penalty decays exponentially with as the look-ahead step k increases.

The total reward configuration is shown as follows:

$$r_t = \begin{cases} -1, & \text{if any}(d_{ASs}) < d_{\text{coll}} \text{ or any}(d_{ADs}) < d_{\text{coll}} \\ -F_{\text{scale}}|d_t^g| + r_{\text{time}} + r_{\text{MLP}}, & \text{otherwise.} \end{cases} \quad (5)$$

where $d_{ASs} = [d_{AS1}, d_{AS2}, \dots, d_{ASm}]$ denotes the Euclidean distance vector between the AGV and observed static obstacles $\{1, \dots, m\}$ in its FOV. $d_{ADs} = [d_{AD1}, d_{AD2}, \dots, d_{ADn}]$ is the relative distance vector between the AGV and other dynamic obstacles $\{1, \dots, n\}$ in its FOV. d_{coll} is the collision distance threshold, which is empirically set to 0.1 m for a moderate effect. When the AGV's distance to any static obstacles (d_{ASs}) or dynamic obstacles (d_{ADs}) falls below the collision threshold d_{coll} , the reward imposes a severe fixed penalty of -1 . This penalty directly discourages unsafe behavior by reducing the policy's expected return whenever collision risk is imminent, forcing the AGV to prioritize maintaining safe distances.

$d_t^g = \|p_t^g - p_t\|_2$ denotes the relative L2 distance between the target point and the AGV at timestep t . $F_{\text{scale}} = 1/R_{\text{env}}$ is the scaling constant. R_{env} is the radius of the current scene. $r_{\text{time}} = -0.01$, given at each timestep, is a constant penalty. This term encourages the AGV to reach the goal as quickly as possible.

Inspired by Liu et al. (2023), we further design a prediction-based multi-step lookahead reward function r_{MLP} to guide the AGV to learn a more reasonable and forward-looking collision-free motion policy in complex dynamic scenarios, handling unexpected situations such as sudden appearance of obstacles. As illustrated in Fig. 3, the MLP reward configuration between the AGV and j th dynamic obstacle is as follows:

$$r_{\text{MLP}}^{\text{AD}_j} = \begin{cases} -e^{-k}, & \text{if } \min_{k=1, \dots, K} [d_{A'D_j}^{t+k}] < d_{\text{com}}. \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

where $d_{\text{com}} = 0.3$ m is the comfort threshold of interaction distance between the AGV and a dynamic obstacle. Note that the $d_{\text{com}} = 0.3$ m is empirically determined and tied to specific environmental constraint. First, considering AGV's physical size of 0.3×0.3 m and maximum operation speed of 1.0 m/s, this d_{com} can provide a 0.1 m safety margin beyond calculated stopping distances of 0.2 m. The effect in such case tends to ensure the AGV can avoid collisions even at maximum speed, preventing abrupt braking that would disrupt trajectory stability. Also, the objectives of this work are environments such as factory or laboratory, where the industrial AGV safety standards typically recommend 0.2–0.5 m minimum distances to obstacles. Hence, $d_{\text{com}} = 0.3$ m falls within this range, gaining a trade-off between safety (avoiding

crowding) and efficiency (preventing unnecessary detours). t is the current timestep. $d_{A'D_j}^{t+k}$ represents the lookahead prediction distance vector between the AGV and the j th dynamic obstacle observed in its FOV. The final MLP reward for the AGV is the minimum of all $d_{A'D_j}^{t+k}$, where $j = \{1, \dots, n\}$ is the index of dynamic obstacles that appear in the FOV of the AGVs.

$$r_{\text{MLP}} = \min_{j=\{1, 2, \dots, n\}} r_{\text{MLP}}^{\text{AD}_j} \quad (7)$$

This penalty decays exponentially with the prediction steps k , meaning an imminent collision is penalized more heavily. This way can discourage shortsighted, jerky maneuvers that might lead to future collision or uncomfortable interaction. Also, the effect in this case tends to ensure that faster dynamic obstacles (which reduce $d_{A'D_j}^{t+k}$ more quickly), trigger stronger and earlier penalty, thus guiding policy to adjust velocity (e.g., slow down, turn) proactively. Note that we rely on two key-points to implement the MLP reward setting in our navigation system. The first one is to predict the trajectory evolution of moving obstacles in FOV at each timestep in the lookahead virtual time domain (LVTd), through exploiting a pre-trained trajectory prediction module, e.g., MID (Gu et al., 2022). Second, for the motion evolution of AGV in LVTd, we choose to perform a uniform AGV state update according to the current evaluated real action pair. This trick speeds up training process, allowing the system to focus on quantifying the policy reasonableness at the actual present timestep.

3.2. ASMAC description

The overall architecture of the ASMAC algorithm for AGV collision avoidance is illustrated in Fig. 2. In the subsequent sections, we elaborate on the implementation details of ASMAC, focusing on the SMRR mechanism, the local attention module, and the new collision-avoidance actor-critic method.

3.2.1. Significance-measured rollout replaybuffer

In the training phase, the interaction experiences generated by a AGV need to be pushed into replaybuffer. Distinct from the previous approach (Everett et al., 2021; Dong et al., 2024) using prioritized experience replay (PER), a novel SMRR mechanism is designed in this work. It utilizes a dynamically-weighted multi-factor significance metric that fuses two complementary potentials, i.e., absolute reward and temporal difference (TD) error. The effect in this way tends to eliminate unreliable early-stage TD errors, and address the PER's limitation of over-relying on a single and static signal. Furthermore, the SMRR

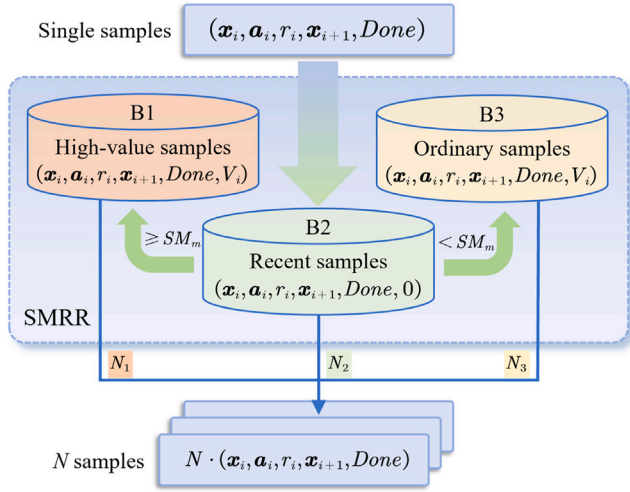


Fig. 4. The structure of significance-measured rollout replaybuffer.

partitions experience sample and store them into different sub-replay pools instead of a single pool, based on their significances. By doing so, this replaybuffer can more frequently rollout the time-efficient and high-significance sample, preventing navigation model from overfitting and facilitates policy learning. The detailed process is as follows.

First, we design a significance measurement standard for classifying experience samples reasonably. The temporal difference (TD) error δ is mostly used for estimating the current state significance in RL. The magnitude of TD-error reflects the extent to which the AGV is able to learn from experiences. There is a great opportunity for improvement towards the expected action, when a larger $|\delta|$ value, since the AGV can timely examine the learning effectiveness of current policy through replaying this type of high-value data. However, the immature policy model at the early stage of training has limited environment exploration, which may introduce a large $|\delta|$ at the edge of the state space to change the convergence direction of network. Compared with δ , the reward r can directly provide feedback on the quality of the current action taken. In the short term, the samples with high reward value have higher learning significance in the early stage of training, which can promote the AGV to quickly learn the collision-free navigation policy in the early stage. Thus, this work builds a new significance measurement method by fusing δ and r . It is formulated as:

$$SM_j = \alpha |r_j| + (1 - \alpha) |\delta_j| \quad (8)$$

where SM_j is the measured significance value of the j th sample. $|r_j|$ is the absolute value of the reward for sample j . α is the weight of $|r_j|$. $|\delta_j|$ is the absolute value of the TD error of sample j . α is dynamically updated with the training process. The value of α is first set to 1 in the early stages of training, which can leverage the reward to guide the AGV to learn a collision avoidance policy quickly. With the further training of the model, α gradually decreases to increase the weight of $|\delta_j|$ to better guide the policy update through the TD error.

Once the significance of experience samples has been measured, they are partitioned and stored in distinct sub-replay pools. As Fig. 4 shows, the SMRR comprises three sub-replay pools: B1, which is designated for storing high-value (significance) samples; B2, intended for recent samples; and B3, utilized for ordinary samples. Each of these sub-replay pools employs a last-in-first-out queue as its storage container and has a predetermined capacity. When the queue reaches its maximum limit, the oldest samples are removed from the head of the queue. The following are the detailed construction steps of the SMRR.

Step 1 (Initialization): The average value SM_m of all samples in the two sub-replay pools B1 and B3 is set to 0, while the weight coefficient α of the reward is set to 1. In the process of model training,

each sample generated is added to the end of queue B2 in turn. When B2 reaches its capacity limit, the oldest sample $(x_i, a_i, r_i, x_{i+1}, Done)$ is ejected from the head of B2 and the value of the ejected sample is calculated as SM_i . If SM_i is higher than SM_m , the ejected sample is added to B1, and otherwise stored in B3. Once the storage of all sub-replay pools is completed, the SM_m and α are updated, and the above process is repeated until the end of the training.

Step 2 (Stratification Sampling): In order to ensure both timeliness and diversity of samples in RL, the stratification sampling strategy is devised in this work to randomly extract appropriate samples from sub-replay pools B1, B2 and B3. This way can create a balanced and representative minibatch for model training. We record the total number of the minibatch as N (1024 herein), including N_1 samples in B1, N_2 samples in B2, and N_3 samples in B3, that is:

$$N = N_1 + N_2 + N_3 \quad (9)$$

where N_2 is set as a fixed value to ensure that each batch sampling can always contain recent samples. This is important because recent experiences often provide valuable information about the current dynamics of the environment. Including them in the training process can help the RL model adapt and update its weights to reflect the most up-to-date knowledge. It is worth noting that N_1 is set to a large value initially, which means that a large number of high-value samples are selected in the early stages of training. The effect in such a case tends to encourage the network to converge in the right direction by emphasizing the most valuable experience. With the progress of policy learning, whereas the performance of the AGV collision avoidance becomes more and more excellent, the occurrence frequency of high-value samples tends to increase. To avoid overfitting with respect to high-value samples, we properly reduce N_1 and increase N_3 as training progresses. This adjustment can make the minibatch more balanced to boost the model generalization.

3.2.2. Local attention module design

The local observation within limited FOV of AGV encompass diverse information that is clearly of different importance to it. Hence, the local attention module is an essential component of ASMAC, which learns weighted local encodings to help AGVs focus on the key features of local observations. As shown in Fig. 2, the multi-head local attention mechanism (LAM) is introduced into this module to compute independent attention weights for same input sequences, which can enable the AGVs to rapidly prioritize critical and changed environmental features. This is a core advantage over traditional methods (Seo et al., 2019; Pham and Pham, 2018) that handle all observation results equally, leading to delay or misfocus. Also, unlike existing interaction graph attention-based methods (Dong et al., 2024; Liu et al., 2023) treating environment observation as a set of unstructured tokens (e.g., grid cell, LiDAR points), this LAM can explicitly categorize observations into distinct entity classes, and process each class with dedicated pre-encoding network. It can ensure that the attention is focused on semantically meaningful entities rather than raw, unclassified data, thus capturing critical information of the environment effectively. Furthermore, distinct from using global attention in Song et al. (2024) requiring weight calculation with pairwise token interactions, this LAM uses dense synthesizer attention mechanism to compute weight, which can greatly reduce computation complexity and preserve key feature extraction. This way lays an important basis for subsequent real-time collision-free robot navigation by using the on-board CPU system for various dynamic scenes. The detailed process is as follows.

The encoding process in this work can be divided into two stages: the pre-encoding stage and the attention encoding stage. To focus on distinguishing and handling different categories of obstacles (entities), we set up individual pre-encoding networks for each entity class in ASMAC. Each pre-encoding network consists of a LayerNorm Layer, a fully connected network (Embedding layer) and a leaky ReLU. The specific process of the pre-encoding stage is as follows.

First, the pre-encoding of AGV self is achieved by using the operation in Eq. (10), that is:

$$e_{self} = f_{self}(x_{self}) \quad (10)$$

where f_{self} is the self pre-encoding network, and x_{self} denotes the information vector of the AGV itself (including self-global velocity and position).

Then, the entity information in the local observation x of AGV is pre-encoded as:

$$\begin{aligned} e_{entity}^{c_1} &= f_{entity}^{c_1}(o_{entity}^{c_1}) \\ e_{entity}^{c_2} &= f_{entity}^{c_2}(o_{entity}^{c_2}) \\ &\vdots \\ e_{entity}^{c_m} &= f_{entity}^{c_m}(o_{entity}^{c_m}) \end{aligned} \quad (11)$$

where c_m ($m = 1, 2, \dots, M$) denotes the various types of entity class. In our robot collision avoidance scenes, the different types of entity classes contain target landmarks, static and dynamic obstacles, and other robots. Note that the separate pre-encoding networks process only relevant data for each entity class instead of raw sensor inputs. The effect in this way can reduce computational overhead, ensuring the LAM outputs attention weights quickly enough to react to sudden changes.

For each entity class, the number observed by the AGV is different. $o_{entity}^{c_m} = [o_{entity_1}^{c_m}, o_{entity_2}^{c_m}, \dots, o_{entity_k}^{c_m}]$, $k \leq N_m$ is the number of entities of class c_m in the FOV for the AGV. For ensuring constant-sized inputs, N_m is set to 15 herein, which represents the default number of entities of class c_m that the AGV can observe. If the number of entities of class c_m observed by the AGV is less than N_m , the space-occupying zero fill vector is used instead. $o_{entity_k}^{c_m}$ represents the information vector for the k th entity in class c_m . $f_{entity_1}^{c_1}, f_{entity_2}^{c_2}, \dots, f_{entity_k}^{c_m}$ are the pre-encoding networks of other entity classes. Thus, through the encoding module, the critical information of the environment can be effectively extracted, while keeping the encoding dimension for the complex dynamic scene constant.

Then, the attention encoding module is used to acquire the local encoding E_{entity}^{atten} for the AGV. The attention encoding module contains three attention matrices: value matrix W_v^{entity} , key matrix W_k^{entity} , and query matrix W_q^{self} . Similar to the pre-encoding networks, each type of entity is equipped with an individual key matrix and value matrix. However, because we only need to measure the contribution of these entities to AGV decision-making, the AGV itself is only equipped with the query matrix, in order to reduce the computation. The pre-encoding is mapped through the attention matrix, as below:

$$\begin{aligned} Q &= W_q^{self} e_{self} \\ K_{entity}^M &= W_k^{entity} E_{entity}^M \\ V_{entity}^M &= W_v^{entity} E_{entity}^M \end{aligned} \quad (12)$$

where $E_{entity}^M = [e_{entity}^{c_1}, e_{entity}^{c_2}, \dots, e_{entity}^{c_M}]$ denotes the pre-encoding set of the various entities for the AGV. Finally, the local encoding E_{entity}^{atten} is obtained by dot product weighting, i.e., Eq. (13). It integrates only the critical environmental information and is input to the actor-critic networks to generate collision-free motion commands.

$$E_{entity}^{atten} = \alpha_{entity} \cdot V_{entity}^M \quad (13)$$

where the attention weight α_{entity} measures the correlation between various types of pre-encoding E_{entity}^M and the AGV, generating the final local feature encoding for the actor-critic network. Herein, α_{entity} is calculated by a dense synthesizer attention mechanism, that is:

$$\alpha_{entity} = \text{Softmax}(F(Q \cdot [K_{entity}^M]^T)) \quad (14)$$

where the structure solely utilizes a two-layer fully connected network F and Softmax to calculate the attention weights. Specifically, $Q \cdot [K_{entity}^M]^T$ computes the raw correlation between the AGV's query and

each entity's key. $F(\cdot)$ nonlinearly models the importance of each correlation, enabling the navigation model to learn task-specific attention patterns. Softmax($\cdot$) normalizes the output of $F(\cdot)$ to ensure the sum of weights for all entities is 1, converting correlation scores into interpretable attention weights. This way helps the model quickly learn important information from the local observations and does not require interaction between tokens (common in global attention Song et al., 2024). Thus, through the local attention module, the AGVs can better evaluate the critical information of the local observation and make more accurate collision-free navigation decisions. Also, it is noteworthy that the local attention module in this work is trained implicitly through the overall reinforcement learning objective, not with an explicit auxiliary reward for attention. Hence, this local attention module does not have a separate attention reward equation.

3.2.3. New collision-avoidance actor-critic method

The existing actor-critic method for collision avoidance navigation faces many problems, e.g., unstable update performance, low generalization ability, and weak exploration efficiency. To address these issues, the PPO and AM are extended to the navigation system to further enhance the ASMAC model.

With the integration of PPO, the AGV gains an extra reward proportional to policy entropy at each timestep. This changes the optimization objective to:

$$\arg \max_{\pi} \mathbb{E}_{(x_t, a_t) \sim \pi} \left[\sum_{l=0}^T \lambda^l (r(x_t, a_t) + \eta H(\pi(a_t|x_t))) \right] \quad (15)$$

where η is a temperature coefficient that is used to make a trade-off between maximizing policy entropy and maximizing reward. The policy entropy, $H(\pi(a_t|x_t)) = -\log \pi(a_t|x_t)$, can measure the policy randomness and make the algorithm more generalizable and robust. The optimization term, $\arg \max_{\pi} \mathbb{E}_{(x_t, a_t) \sim \pi} \sum_{l=0}^T \lambda^l r(x_t, a_t)$, is used to maximize environmental reward. Through maximizing policy entropy, the approach enhances the exploration ability of the navigation policy and possesses stronger robustness and generalization.

The optimization goal is achieved by PPO training paradigm, and the detailed process is shown in Fig. 2. The PPO is an actor-critic method based stochastic policy gradient. It introduces importance sampling trick (He et al., 2022) to separate the target policy from the sampling policy for estimating policy gradient. In this way, the estimation of policy gradient for PPO in an on-policy manner can be changed to an off-policy mode. Specifically, when the off-policy importance sampling strategy is used for gradient calculation, the objective function for the PPO's actor network can be formulated as follows:

$$\mathcal{L}^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)) \hat{A}_t \right] \quad (16)$$

where $r_t(\theta) = \pi_{\theta}(a_t|E_{entity,t}^{atten}) / \pi_{\theta_{prior}}(a_t|E_{entity,t}^{atten})$ is the ratio between the probability of an action under the current policy $\pi_{\theta}(a_t|E_{entity,t}^{atten})$ and the probability of an action under the prior policy $\pi_{\theta_{prior}}(a_t|E_{entity,t}^{atten})$. In the process of updating parameters, the policy setting of PPO is divided into one prior (behavior) policy $\pi_{\theta_{prior}}$ and one current (target) policy π_{θ} . The $\pi_{\theta_{prior}}$ is exploited to interact with the environments for collecting the training samples, while the π_{θ} performs parameter optimization based on sampled data from $\pi_{\theta_{prior}}$ to generate optimal action commands. In this case it means that the learning is from data "off" the target policy, and the overall process is termed off-policy learning. The benefit of this off-policy learning lies in that the target policy parameter can be updated several times based on the sampled data from $\pi_{\theta_{prior}}$ for efficient decision-making. In this perspective, this off-policy learning for the PPO method can make sampled data reusable, thus offloading the expensive online computation to an offline training procedure. Besides, a clip mechanism is introduced by OpenAI (Schulman et al., 2017) to limit the change of $r_t(\theta)$ in a range between $1 - \epsilon$ and $1 + \epsilon$, where ϵ is the hyper-parameter belonging to $[0, 1]$. By doing so, the distributions of θ and θ_{prior} differing too much can be avoided

while preventing the gradient from taking an excessive step size. The reason behind this design is that too large step size for the gradient may introduce a big learning rate that would produce problem with convergence difficulty. $\hat{A}_t$ represents the estimated advantage at the time-step t , which is calculated using the generalized advantage estimation (GAE) technique (Schulman et al., 2018). GAE exploits an exponentially weighted average to acquire a good trade-off in terms of the deviation and the variance. It is formulated as:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \left[\gamma V_{\omega}(\mathbf{E}_{\text{entity},t+l+1}^{\text{atten}}) - V_{\omega}(\mathbf{E}_{\text{entity},t+l}^{\text{atten}}) + r_{t+l} \right] \quad (17)$$

where $V_{\omega}(\mathbf{E}_{\text{entity},t+l}^{\text{atten}})$ used as a baseline to estimate the advantage is a state-value function on parameter ω . λ and γ are two hyperparameters that can be utilized to control the magnitude of the variance, r_{t+l} is the reward value at time-step $t+l$.

The critic network is an approximation function of baseline $V_{\omega}(\mathbf{E}_{\text{entity},t+l}^{\text{atten}})$. The calculation process of the mean square error loss is as follows:

$$\mathcal{L}^v(\omega) = - \sum_{t=1}^T \left(\sum_{t'=t}^T \gamma^{t'-t} r_{t'} - V_{\omega}(\mathbf{E}_{\text{entity},t}^{\text{atten}}) \right)^2 \quad (18)$$

It is noteworthy that the parameter θ and ω are updated in each epoch with learning rate lr_{θ} and lr_{ω} , respectively, based on the Adam optimizer (Adam et al., 2014).

To facilitate the overall learning efficiency, the AM mechanism (Mnih et al., 2016) is adopted for policy learning. As shown in Fig. 2, the entire architecture consists of a global PPO and several independent PPO workers. In each process, the PPO worker has the same network structure as the global PPO, and individually interacts with each navigation environment to acquire experience data. Unlike existing synchronous method (Fan et al., 2020) assuming that n PPO workers are acting in a fully synchronous manner for environment exploration, our AM mechanism allows n PPO workers to operate independently without interference with each other. In this sense, this AM mechanism can make each PPO worker store its own gradient data in a separate cache and periodically push the cached data to the global PPO for updating the parameters of ASMAC. Concretely, when each PPO worker interacts with its own environment to obtain experience data, it calculates a separate gradient of loss function in its own thread at each step, followed by uploading the gradient to the global PPO. Once all gradients computed by n PPO workers are accumulated, the global PPO performs the gradient descent to update the parameters of its actor and critic networks. Once this update is completed, these new parameters are sent back to all independent PPO workers simultaneously to complete an asynchronous update. At this point, all independent PPO workers share the same updated parameters. Then, each PPO worker uses the parameters of global PPO to recompute the advantage $\hat{A}$ and update the old policy π_{θ} to continue interacting with environments. Whereas all PPO workers share same parameters, their interactions with the environments are independent. This allows the PPO worker to explore different parts of the state space and gather the diverse experience data, which can improve the learning process when aggregated by the global PPO. Also, since each PPO worker only use global parameters for action execution and gradient calculation instead of update its parameters independently, all PPO workers can align with the latest policy post-synchronization. Note that all the PPO workers are initialized with same network architecture as global PPO. This ensures workers and the global network process observation and compute policies in a unified manner. At the start of training, all PPO workers inherit the initial parameter of global PPO's actor and critic networks. These parameters are randomly initialized and remain consistent across workers, avoiding bias from divergent initial states. Also, the quantity of PPO workers used in the AM architecture is empirically set to 8 for a moderate trade-off between computation efficiency and exploration diversity. The goal position and starting point of the AGV in each navigation environment

are randomly set for boosting model generalization. It is concluded by the above AM architecture modeling that independent PPO workers share parameters with the global PPO and with each other. With this AM training setting, our ASMAC method can not only avoid the use of inefficient replay buffer (Mnih et al., 2015), but also break the correlation between training samples, thus boosting learning efficiency.

4. Experiments

To verify the effectiveness of our proposed ASMAC navigation model in avoiding collision under complex dynamic workspaces, we implement our method with ROS Kinetic and Gazebo9.0, while building the policy model based on PyTorch. It is trained on a PC with Intel Core i7-9800X CPU and GeForce RTX 2080Ti GPU. The training process and experiment results of our model are discussed in detail below.

4.1. Model and training details

4.1.1. Model details

The local attention module consists of pre-encoding module and the attention encoding module. The number of pre-encoding networks in the pre-encoding module is the same as the number of environment entity classes. All pre-encoding networks contain a fully connected (FC) layer with the size dimension [2, 128], a nonlinear activation function (leaky relu), and a LayerNorm layer. The attention encoding module receives pre-encoding cues that include three attention matrices ($\mathbf{W}_k$, $\mathbf{W}_q$, and $\mathbf{W}_v$) with the unit dimension [128, 128]. The three attention matrices are randomly initialized as the same unit dimension of [128, 128], through using the Xavier initialization trick (Dong et al., 2024). This way is able to scale random weights by inverse square root of input dimensions, ensuring the variance of attention outputs remains consistent across layers and preventing vanishing or exploding gradients in early training. Meanwhile, the attention matrices are learned parameters that are updated during the overall training process of ASMAC model. Hence, they are updated via backpropagation, utilizing the Adam optimizer with learning rate 2×10^{-4} . It is noted that only the AGV self has the query matrix, and the number of $\mathbf{W}_k$ and $\mathbf{W}_v$ is equal to the observed entity types. The actor-critic module consists of the value network and the policy network. The value network receives the local encoding from the local attention module, which contains two hidden layers with dimensions [128, 128] and [128, 1]. The structure of policy network is similar to value network. It also takes the local encoding as the input, and consists of three hidden FC layers with the dimension of [128, 128], [128, 128], and [128, 2], respectively. Note that the hidden layers of the actor-critic networks are equipped with leaky relu as nonlinear activation function to mitigate the vanishing gradient problem during training. The specific parameter settings of ASMAC are presented in Table 2.

4.1.2. Training details

In order to learn a robust collision avoidance policy, we expose our AGV to diverse workspaces by creating different complex scenes with various obstacles. These simulation scenarios are built using the Gazebo mobile robot simulator (Koenig and Howard, 2004) to accomplish the policy training, as shown in Fig. 5. For scenario 1 without any obstacles, we generate random start and goal positions for robot at the beginning of each episode. Robot in scenario 3 is randomly initialized to avoid the static obstacles. As for scenarios 2 and 4, the policy is trained to realize autonomous navigation for the robot in a variety of pedestrian crowds. Also, we add timestep-varying random noises to the local observation of AGV to simulate imperfect sensing environments. This noise mimics partial sensor failures (e.g., missing obstacle position data), which forces the policy to learn robust feature extractions.

Whereas RL training in various scenarios simultaneously can enable robust performance of collision avoidance, it enables the training procedure more difficult. To tackle such difficulty, a two-stage training

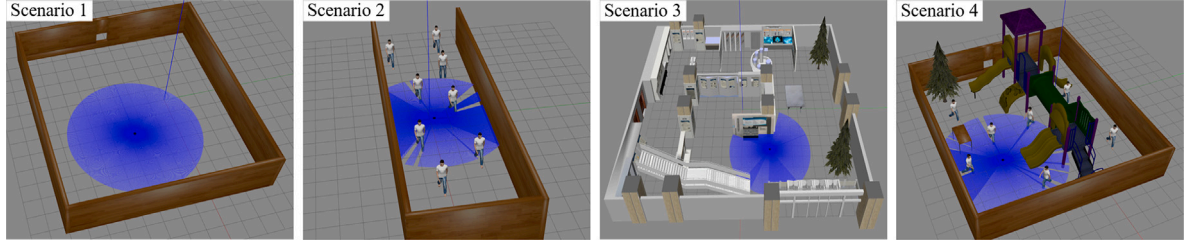


Fig. 5. Scenarios used to train the collision avoidance policy.

Table 2

Parameter variable settings of our ASMAC navigation model.

Parameters	Value
PID gains K_p, K_i, K_d	0.2, 0.1, 0.02
K (MLP)	5
Comfort threshold d_{com}	0.3 m
Num entity type M	4
Num entity N_m	15
Num PPO worker n	8
Num episode	5×10^4
Num start train	1024
Episode length	100
Learning rate lr_θ, lr_w	$1 \times 10^{-4}, 2 \times 10^{-4}$
Buffer size	2×10^5
Batch size N	1024
Num samples N_1, N_2, N_3	600, 256, 168
Hyperparameters λ, η	0.96, 0.01
Update frequency	4
Multi head h	4
FOV	2π , 8 m

process for the ASMAC model is proposed, through the inspiration of curriculum learning paradigm (Bengio et al., 2009). The simulation scenarios in Fig. 5 are used to train the RL-based model in two stages, from simple to complex. In the first stage, our goal is to encourage AGV to learn a high-efficiency collision avoidance pattern. For that, we only train the robot in a simply scenario (i.e., scenarios 1 and 3 in Fig. 5) with static obstacles. The effect in such case tends to quickly find a reasonable collision-free solution. Once the AGV achieves reliable level of collision avoidance, this training process is stopped and the trained policy is saved. Then, we continue to update the policy in the second stage by training in other more complex scenarios 2 and 4 in Fig. 5, while exploiting a larger collision penalty (-6) to ensure the navigation safety. Clearly shown in Fig. 6, the two-stage training policy can get higher rewards and has faster convergence speed than the policy trained from scratch with the same number of episodes. It is noteworthy that when switching to second training stage, our reward curve drops slightly because of the change of train scenario, but the curve rises to a higher point very soon. The reason behind this may lie in two aspects: (1) The changes of training scenarios can introduce new variations. This can make the model gained from the first stage to struggle to adapt to the unfamiliar condition initially, leading to a temporary drop in performance. (2) The adjustment of hyperparameters in model during the transition to the second training stage could also have a slight negative impact on its learning process, but it can ultimately lead to better improved performance once the model captures the underlying pattern in the new data.

4.2. Evaluation of our navigation model with different configurations

Our navigation policy exploits a combination of local attention (LA) and significance measurement (SM), and is augmented with MLP, PPO, and AM. In the following part, an intensive experimental analysis of our

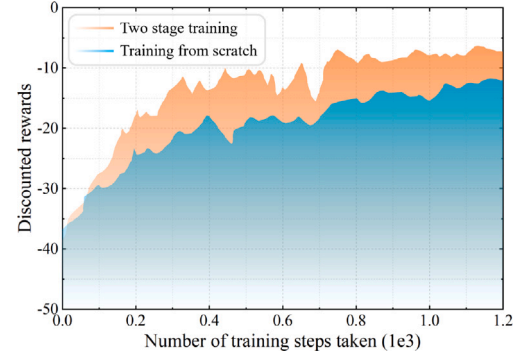


Fig. 6. Discounted reward curves for each step during the training process.

Table 3

Abbreviations of the proposed different configuration models.

Abbreviations	Configurations
PPO (actor-critic)	RL-only navigation model, i.e., baseline
LPPO	Integrate local attention module (LAM) in PPO
SLPPO	Fuse SMRR in LPPO
MLPPO	Integrate MLP in LPPO
ASMAC (Ours)	Integrate AM mechanism in MLPPO
Ours-NoPID	ASMAC without PID operations

navigation model with different configurations is built to systematically ablate the components and review the importance quantitatively. To simplify the expression, the abbreviations of proposed model with different configurations are given in Table 3. Note that PPO, LPPO and SLPPO all utilize the basic configuration (without MLP reward) as their reward setting, while the other three models use the full reward configuration we describe above (i.e., Eq. (5) that integrates the MLP reward function into the basic configuration). For presenting a fair comparison, all the models are trained on the scenarios shown in Fig. 5 with the proposed two-stage training method. The key difference among these different configuration models is that unlike PPO, LPPO, SLPPO, MLPPO acting in a fully synchronous manner for environment exploration, the ASMAC and Ours-NoPID utilize AM mechanism to enable parallel experience collection and gradient computation in a separate cache, breaking sample correlation and accelerating exploration. To test above-mentioned models with different configuration, a representative collision avoidance scene shown in Fig. 7 is constructed. In the scene, the multi-moving obstacles at a constant speed are set to encounter the AGV. In detail, the translational speed of obstacle 1 (OBS1), OBS2, OBS3, OBS4, and OBS5 are 0.2 m/s, 0.4 m/s, 0.6 m/s, 0.8 m/s and 1.0 m/s respectively. The pink straight line in the Figure is the reference mission path (RMP) of AGV.

As can be observed, the baseline PPO struggles to guarantee the optimality of final track. This happens because they are rarely able to understand collision situation in the dynamic scenes, resulting in useless or even erroneous searches in wrong direction, especially when

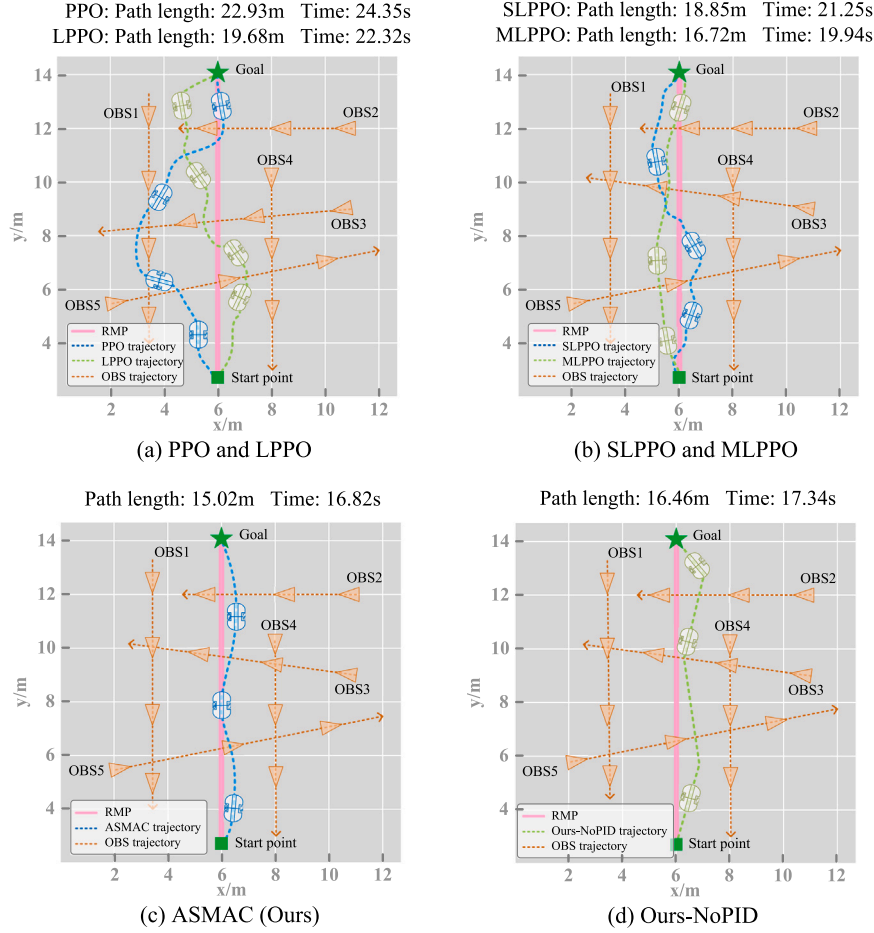


Fig. 7. AGV collision avoidance of the proposed different configuration models in dynamic scene.

obstacles occur. The LPPO can acquire a relatively short track along RMP. This mainly benefits from the LAM that can extract and encode critical environmental features to reduce unnecessary search and improve navigation efficiency. When the SMRR is integrated into the navigation policy, the length of the track generated by SLPPO is shorter than that of LPPO. Yet, the desired collision-free action commands output by LPPO and SLPPO are unstable, leading to significant fluctuations in the AGV's trajectory. It can be found that the track fluctuation can be further mitigated by MLPPO, compared with the SLPPO. This demonstrates the effective function of MLP technique in helping AGV generate more reasonable and farsighted collision avoidance policies. Intuitively, our final ASMAC acquires the best navigation result in the dynamic scene. It can not only complete collision avoidance tasks efficiently, but also produce a continuous, smooth, and dynamic feasible robot trajectory. Besides, the track of Ours-NoPID produces obvious trajectory catastrophe throughout the AGV collision avoidance, which does not satisfy the motion constraints of AGV in practical applications.

While it is easy to find interesting pros and cons for any given navigation task, it is more useful to draw a conclusion after aggregating a large number of randomly-generated cases. Consequently, we design an algorithm to randomly generate 500 navigation episodes for the scene to quantitatively evaluate the performance of different configuration models. To quantify the navigation performance, a set of evaluation metrics are adopted in the experiments, including success rate (SR), time to goal (TG), collision rate (CR), discounted reward (DR) and trajectory length (TL). Note that the SR means that rate of case in

Table 4

Performance metrics comparison of different configurations on the dynamic scene.

Methods	SR	CR	Avg TG (s)	Avg DR	Avg TL (m)
PPO	0.52	0.65	31.36	-25.13	23.15
LPPO	0.66	0.43	22.46	-21.65	19.67
SLPPO	0.79	0.32	20.62	-18.74	18.12
MLPPO	0.92	0.09	18.95	-13.86	17.48
Ours	0.98	0.02	16.22	-11.52	14.95
Ours-NoPID	0.96	0.04	17.56	-10.38	16.32

which AGV successfully reaches its target within 60 timesteps. The CR is defined as the percent of cases with a collision. TG and TL are reported based on the success cases. Detailed comparison results are aggregated in Table 4.

First, we observe a relatively better navigation performance for the LPPO model by comparing the results from PPO. It verifies the effect of LAM in assisting our method to focus on the information that is more important for collision-free action decision. Subsequently, considering the navigation performance difference among LPPO, SLPPO and MLPPO, we see that the idea of each component has a positive effect on the model. When bringing the SMRR into LPPO, the navigation results of SLPPO are significantly superior to that of LPPO. It achieves 0.79 in SR and 20.62s in average TG, showing 19.7% improvement and 8.19% reduction in average SR and TG respectively. When MLP and SMRR are both integrated into LPPO, the average TG of MLPPO is a

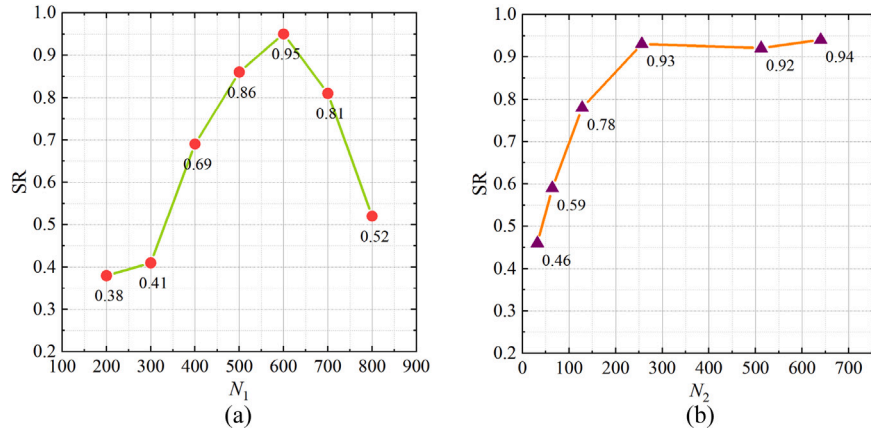


Fig. 8. The curves of SR with different (a) N_1 and (b) N_2 parameters on the dynamic scene of Fig. 7.

gentle reduction with respect to SLPPO, while the SR has a significant ascent from 0.79 to 0.92. We consider that the SMRR enhances the sample utilization that is conducive to limit the sampling space for efficiency while increasing the probability of avoiding collision to some extent. Also, the designed MLP generally motivates AGV to master the skill of generating more reasonable collision avoidance decisions. It is critical for collision-free policy learning in complex dynamic scenarios.

Clearly, our ASMAC combining both PPO and AM to jointly optimize navigation policy, gains best performance on the complex dynamic scene in almost all metrics. This shows the effectiveness of the combination of each component in improving the performance of the method. Additionally, when comparing Ours with Ours-NoPID, it can be found that our model shows 2.04% improvements in SR, while 50% and 7.6% reductions in CR and average TG, respectively. We suggest that the PID facilitates us introduce kinematics constraints to enhance the learned navigation model, enabling AGV to final continuous, dynamic-feasible, and smooth trajectory travel.

It is noteworthy that while the ASMAC system performs well for obstacles up to 1.0 m/s (the maximum tested speed), there are failure modes observed under fast obstacle motions. In detail, due to the MLP's limited lookahead horizon ($K=5$ lookahead timesteps), the failure mode can emerge under extremely fast obstacle motions (> 1.0 m/s). To address this, future works will be done to develop a local temporal-spatial graph-based social encoder with multi-sensor fusion, which can perform social-interaction hidden state extractions, and handle the issue of failure modes observed under fast obstacle motions within the FOV of AGV.

In order to further analyze the impacts of parameters N_1 , N_2 , and N_3 in collision-free navigation of our ASMAC, a one-factor-at-a-time (OFAT) method is employed, where each parameter is varied within a predefined range while others are held constant at default value, as shown in Table 5. When 500 navigation tasks are randomly generated in the dynamic scenes of Fig. 7, the test results in terms of SR are shown in Fig. 8. The N_1 is used to adjust the proportions of high-value samples to encourage the network to converge in the right direction by emphasizing the most valuable experiences. In Fig. 8(a), the SR value shows a trend of first rising and then falling when N_1 increases. The main reason is that neither too small nor too large value of N_1 can realize a moderate trade-off between high-value and ordinary samples. Since a better navigation performance is achieved when N_1 is changed from 500 to 700, N_1 is set to 600 for a good trade-off in this work. The N_2 is used to maintain recent samples to provide valuable information about the current dynamics of environments. From Fig. 8(b), we can observe that the navigation performance improves gradually when N_2 increases from 32 to 640, while tends to be stable when N_2 is around 256. Hence, N_2 is set as 256 to properly sample recent experiences.

Table 5

Parameter ranges and default values for sampling stratification sensitivity analysis.

Parameters	Range	Default value
N_1	200, 300, 400, 500, 600, 700, 800	600
N_2	32, 64, 128, 256, 512, 640	256

4.3. Collision avoidance performance evaluation

In order to further verify the superiority of our ASMAC in collision-free navigation, we provide a thorough evaluation by comparing it with four mainstream available methods, including RL-based AL-DSAC (Zhou et al., 2024), TL-HDR (Zhang et al., 2024), M-DQN (Sheng et al., 2025) and conventional APF (Malone et al., 2017) methods. It is noteworthy that we utilize the reward function as defined in Eq. (5) for all RL-based methods in this part to present a fair comparison among them. Besides, all the RL-based policy models are trained on the scenarios shown in Fig. 5 with the proposed two-stage training method. The APF method for AGV is implemented by an abstract artificial potential field with the default parameters provided by the author.

In the inference test stage, two types of challenging test scenarios that do not occur during training phase are built in our experiment: (1) “narrow passage” and (2) “detour”. Collision avoidance in the narrow passage is a common issue for AGV navigation. In such case, the passable area is generally located in the gap among two obstacles. The navigation evolution processes with narrow passage are detailed in Fig. 9. Note that we randomly set four pedestrians moving around to increase the difficulty of AGV collision avoidance in narrow passage. Each pedestrian is controlled by the plugin “ActorPlugin” within Gazebo in this study. The results show that it is difficult for the RL-based AL-DSAC, TL-HDR and M-DQN approaches to sample the area within the narrow channel in a limited number of iterations. The limitation makes it hard to plan a reasonable path in the narrow passage. We can observe in traditional method that APF can narrow sampling space based on repulsive force generated when approaching obstacles to shorten useless searches. Yet, it still suffers from the impoverishment of the trajectories swing violently when AGV passes through pedestrian. Also, it can be seen that our ASMAC not only greatly reduces the unnecessary searches but significantly improves the stability and smoothness of trajectories.

Subsequently, the “detouring” scenario is designed to simulate the task of AGV avoiding dynamic obstacles in the course of detour to reach the goal. Herein, the pedestrian is exploited as moving obstacle for experimental evaluation, which walk on a 1.3-meters fixed-radius circle at an angular velocity of 0.4 rad/s. The real-time evolution results are shown in Fig. 10. We observe that our ASMAC can output a reasonable

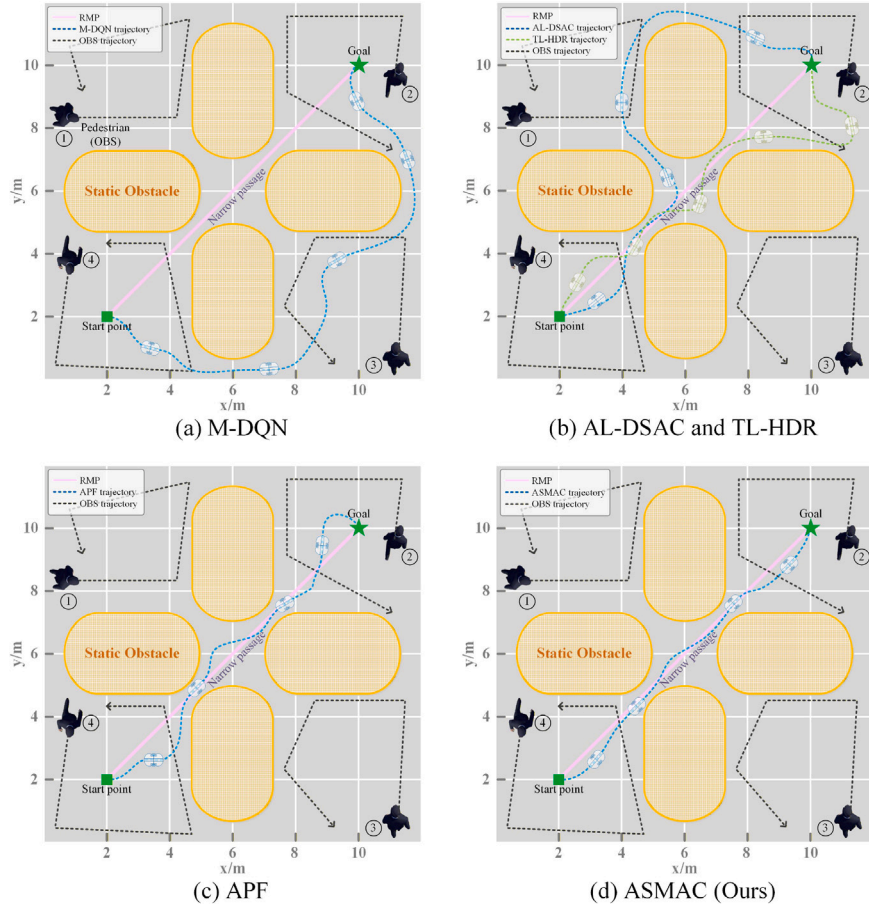


Fig. 9. Collision avoidance navigation of the different methods in the “narrow passage” scenario.

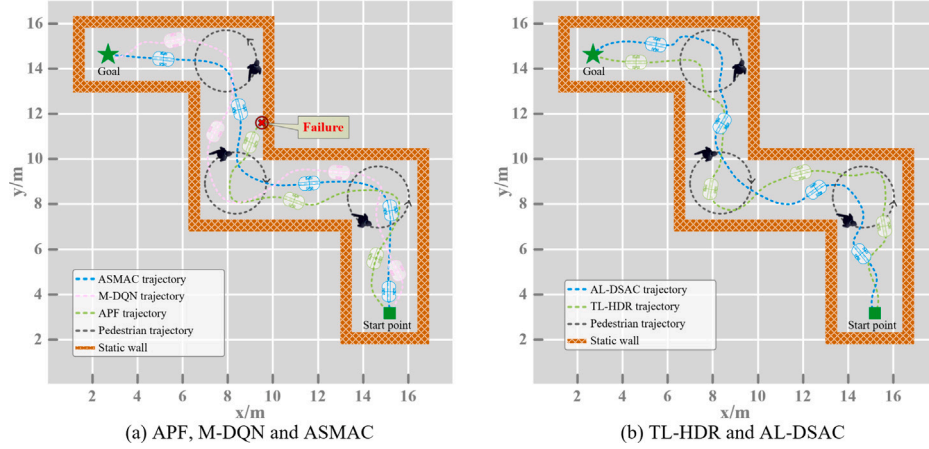


Fig. 10. Collision avoidance navigation of the different methods in the “detour” scenario.

policy, making AGV avoid movement obstacles smoothly and detour to reach the goal successfully. However, the effect in such case tends to bring about great difficulty to other approaches. For instance, AL-DSAC, TL-HDR and M-DQN can avoid collisions, but generating unnatural trajectories obviously. This means that the desirable decision output is unstable that cannot guarantee the trajectory's optimality. Unexpectedly, APF fails to collision avoidance. The potential reason may be that AGV is much less affected by obstacles' repulsive force than by the attraction of goal in such cases, which makes it difficult to plan a detouring path to dodge obstacles based on the APF algorithm.

To further compare the quality of collision-free navigation quantitatively, we summarize SR, CR, TG, and TL for different methods in Table 6, through randomly-generated 600 navigation tasks in each complex scenario. As can be seen, the conventional APF has a low SR, but the navigation time within limited successful cases is short. For the RL-based AL-DSAC and M-DQN, although a general better performance can be achieved in terms of the SR and CR, they consume more time to generate longer collision-free trajectory. The reason may lie in that due to limited training data, these trained RL methods do not yield sensible performance, thus needing to a long-distance navigation for better collision-free exploration. It also shows that TL-HDR achieves

Table 6
Performance metrics comparison of different methods on two challenging scenarios.

Models	Narrow passage (12 m×12 m)				Detour (18 m×18 m)			
	SR	CR	Avg TG (s)	Avg TL (m)	SR	CR	Avg TG (s)	Avg TL (m)
APF	0.65	0.35	18.53	18.36	0.16	0.85	24.35	27.62
M-DQN	0.74	0.27	26.63	22.58	0.72	0.39	29.62	33.78
AL-DSAC	0.82	0.19	25.87	21.96	0.81	0.20	28.51	32.39
TL-HDR	0.93	0.07	17.96	18.02	0.92	0.08	23.27	26.98
ASMAC (Ours)	0.97	0.03	16.45	17.62	0.98	0.02	21.15	25.18

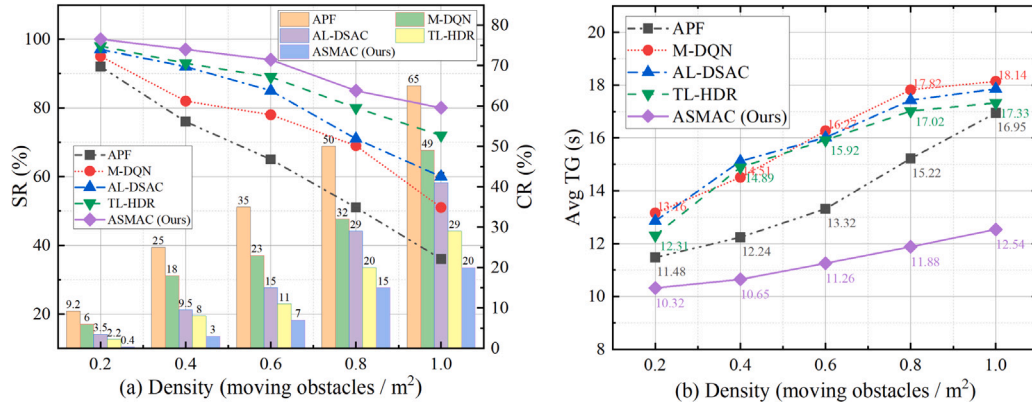


Fig. 11. Navigation performance (a) SR, CR, and (b) average TG comparisons for different methods in scenes with varied moving obstacle densities.

a suboptimal performance. This mainly benefits from the maximum causal entropy-based inverse that can capture the human preference characteristics of low-risk areas of the environment. In summary, our ASMAC achieves best navigation performance on these two scenes for all metrics. It is mainly attributed to three aspects: (1) ASMAC uses the combination of AM and PPO as the underlying actor-critic architecture instead of RL-purely, for efficient and robust decision-making, and considers PID operation, which lays a foundation for final executable, smooth, and continuous navigation. (2) ASMAC uses the MLP trick to predict the future events of collision threat. It can not only evaluate the impact of current action on future AGV-environment interaction, also prevent AGV from generating intrusive, shortsighted, and unnatural motion policies. (3) Through the tight cooperation between SMRR and LAM, ASMAC system can improve its perception ability, thus making effective and safe collision avoidance decisions rapidly under complex dynamic scenes.

Moreover, we design one extra scene with varied moving obstacle densities to further evaluate the performance of different methods. Specifically, this scene involves a 7×7 m² region, allocating 10, 20, 30, 40, and 50 obstacles that randomly move around, respectively. Thus, the obstacle density for the scene is about 0.2, 0.4, 0.6, 0.8 and 1 moving obstacle per square meter, respectively. Each obstacle density of evaluation involves 800 different navigation tasks. From the experimental results in Fig. 11(a), whereas all methods have a decline in SR and a rise in CR when the density changes from 0.2 to 1, our ASMAC by a large margin outperforms the other methods. As shown in Fig. 11(b), ASMAC has least time-consuming. The navigation time of our method only increases gently from 10.32 s in 0.2 density to 12.54 s in 1 density, while that of others has a dramatic rise, e.g., APF from 11.48 s to 16.95 s. It is concluded by the data comparison that our ASMAC can make effective decision to safely complete collision avoidance in complex dynamic scenario, still preserving high real-time performance.

4.4. Multi-AGV collision avoidance test

In the above experiments, we focus on testing the collision-free navigation of a single AGV. In this section, we design multiple AGVs

operating independently to avoid each other's experiments to further examine the navigation ability of our approach. In detail, an 8-AGV bilateral bidirectional interaction scenario is developed, in which each one of these AGVs utilizes same method for avoiding other AGVs during interaction. In the experiment, we adopt M-DQN, AL-DSAC and TL-HDR as comparison baselines. The ultimate multi-AGV interaction results are shown in Fig. 12, including the comparison of total path length and average timestep of AGVs. As we can see, the interaction results for both M-DQN and AL-DSAC intuitively look similar, with AL-DSAC being slightly more efficient. Besides, each AGV trained by the two methods are relatively conservative, tending to in advance go around a distance. Contrarily, each AGV using our ASMAC can acquire a proper track for interaction to guarantee no collision with other AGVs, while achieves best results in terms of the average timestep and total path length metrics. This suggests the effectiveness and robustness of our ASMAC in multi-AGV collision avoidance. It also shows that the interaction results of TL-HDR are not better than that of ASMAC. The potential reason is that due to limited perception data from single sensor (LiDAR), TL-HDR cannot capture critical environmental feature effectively, thus preventing the AGV from focusing on important collision-free policy exploration.

Also, we set the AGV number to 16 in this bilateral bidirectional interaction scenario, which is a highly-challenging scene to collision avoidance. M-DQN, AL-DSAC and TL-HDR are also used as three comparison baselines. When 500 navigation tasks are randomly generated in this scene, the SR, CR, TL and TG metrics are reported in Table 7. As can be seen, M-DQN has worse navigation results than AL-DSAC and TL-HDR, while our ASMAC remains the best in navigation success, safety and efficiency among all four competing methods. This confirms again the effectiveness and scalability of our ASMAC method in collision avoidance.

4.5. Real-world navigation analysis

The ASMAC method presented in simulations has realized many satisfactory properties, but it still exists some challenges of transferring our learned policy ASMAC from simulation to real world. First, in real situations, noises in observation are ubiquitous due to imperfect

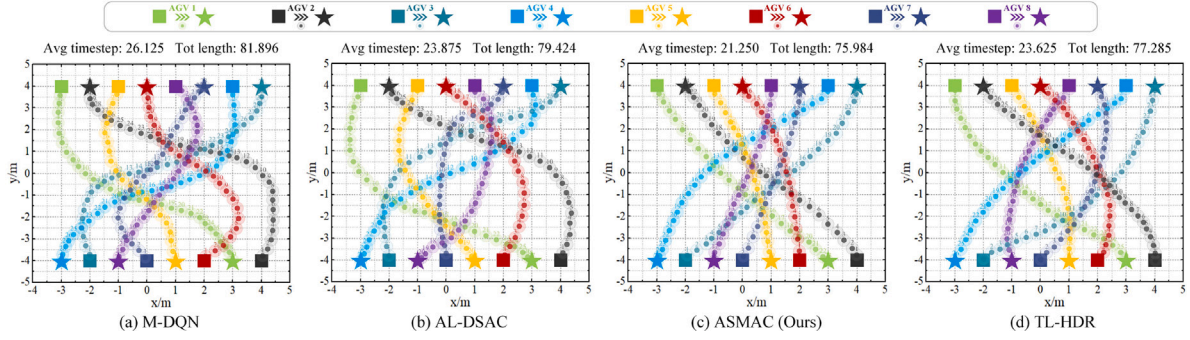


Fig. 12. Visualization of eight-AGV bilateral bidirectional interaction results of different methods.

Table 7

Performance comparison of collision-free navigation of different methods on sixteen-AGV bilateral bidirectional interaction scenarios.

Methods	SR	CR	Avg TG (s)	Avg TL (m)
M-DQN	0.62	0.45	25.12	12.62
AL-DSAC	0.78	0.26	21.46	11.83
TL-HDR	0.89	0.12	19.53	10.96
ASMAC (Ours)	0.98	0.02	17.61	9.98

sensing. Second, the simulated ASMAC can adjust velocity smoothly by PID, but real AGV with low-torque motors may fail to decelerate or accelerate quickly enough to avoid dynamic obstacles, e.g., a fast-moving pedestrian. Finally, it is not guaranteed that different real AGVs generate consistent behavior given same control command, due to many real-world factors, e.g., motor properties, mechanics details, and frictions cannot be accurately modeled in simulation.

To further test the generalization capabilities of our navigation model in real world, we use a self-developed differential-driving robot prototype to demonstrate our model, as shown in Fig. 13. The sensors are a Rplidar A2 Lidar (used for obstacle detection), a Kinect V2 camera (used for moving object detection Sun et al., 2023), and a NOKOV motion capture system (used for indoor global localization). These sensor kits provide the observations for AGV. Then, AGV executes action commands output from our navigation system that consists of a software framework (ROS Kinetic in Ubuntu16.04) and an industrial computer (Intel Core i7-9800X CPU 8-GB RAM). Although there is no perfect state knowledge, as was available in simulation, the AGV can avoid collisions using only on-board sensing.

Note that in the practical implementation of the proposed ASMAC framework, the observation distribution $\mathcal{O}(s)$ formally defines probability of receiving local observations x_t given the true global state s_t in a POMDP. In this work, it is pragmatically approximated via the AGV's sensor fusion pipeline and LAM, which together handles the "imperfect local observation" characteristics of the POMDP for collision-free navigation. Specifically, the AGV's onboard sensors replicate imperfect the local observation scenes assumed in method's design, and collect heterogeneous data that collectively approximates $\mathcal{O}(s)$ by reducing observation uncertainties for the ASMAC model. Meanwhile, by fusing the collected heterogeneous data can reduce individual sensor noise and FOV limitations, thereby increasing the likelihood that observed x_t aligns with true s_t . Furthermore, the local attention module (LAM) refines the approximation through prioritizing collision-critical observation. On the one hand, it utilizes class-specific network to pre-encode observations of distinct entity types. This ensures that observations of high-impact entities are weighted more heavily in final encoding E_{entity}^{atten} , thus effectively sharpening the observation distributions to focus on states that matter for safe navigation. On the other hand, it utilizes the dense synthesizer attention to fuse entity observations with

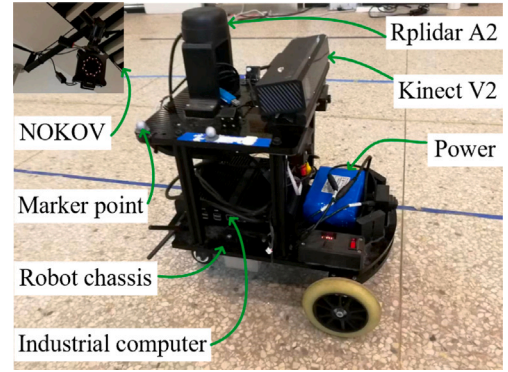


Fig. 13. Robot prototype for navigation experiments of collision avoidance.

the AGV's self-state. This process cannot solely downweight irrelevant observations, but also amplify reliable and relevant ones. The effect in such cases is inclined to approximate $\mathcal{O}(s)$ as a distribution where observations with strong collision-relevant signals are more likely to be trusted.

The navigation experiments are conducted in our laboratory (approx. 150 m²). A typical example of our navigation is shown in Fig. 14(a) that does not occur during the training phase. Both static (boxes) and moving (pedestrians) obstacles are placed between robot's starting position and goal about 3 m away to interfere with the robot. From the trajectory in Fig. 14(b), we observe that robot can not only smoothly avoid static obstacles, but slow down adaptively to wait for the pedestrian. After the pedestrians pass, the AGV starts again towards the goal. Eventually, it successfully and safely navigates to its goal without any collisions.

In addition, the navigation performance is quantitatively measured by randomly setting up another 100 tasks in the laboratory with complex dynamic scene. M-DQN, AL-DSAC and TL-HDR are also used as three counterparts for performance comparison. Table 8 lists the quantitative results, including SR, CR, average TL and TG. A navigation task is regarded as a success only if the robot stops near the goal (e.g., 0.2 m) within 100 timesteps. As illustrated in the quantitative results, M-DQN and AL-DSAC obtain worse navigation results than TL-HDR, while our ASMAC remains the best in navigation success, safety, and efficiency among all four competing methods. It has a higher than 7% SR and a lower than 8% CR, while shows a 11.4% reduction in average TG, compared to the TL-HDR. The trial outcomes demonstrate that our ASMAC model can be well deployed to real robot, presenting an excellent collision-free navigation capability in complex dynamic real-world environments.

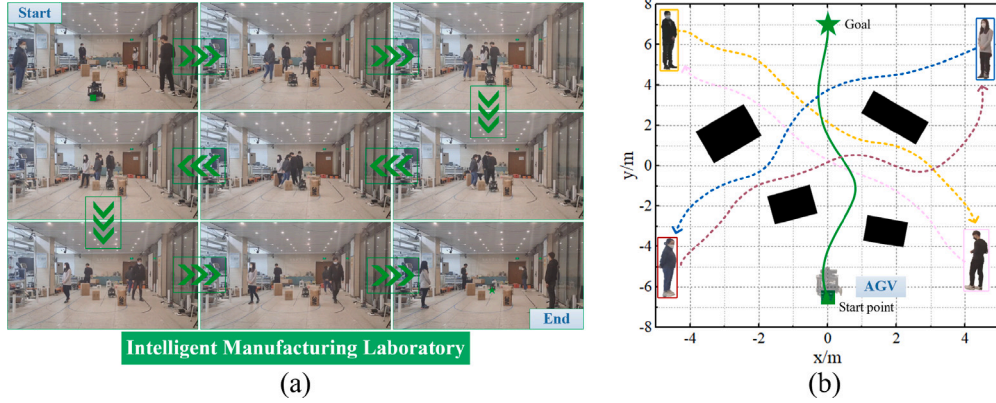


Fig. 14. Collision avoidance navigation of our proposed method in real world (intelligent manufacturing laboratory). (a) the physical robot navigates to its goal in a real complex scene with both moving and static obstacles. (b) the robot's trajectory in the two-dimensional coordinate system, corresponding to (a) on the left. In this Figure (b), the green solid line represents robot's trajectory, the color dashed lines are the pedestrians' paths, and the black boxes refer to the static obstacles.

Table 8

Performance comparison of collision avoidance navigation of different methods on complex dynamic real-world scenarios.

Methods	SR	CR	Avg TG (s)	Avg TL (m)
M-DQN	0.71	0.38	23.16	13.62
AL-DSAC	0.69	0.35	24.28	14.97
TL-HDR	0.90	0.11	19.62	11.83
ASMAC (Ours)	0.97	0.03	17.38	9.96

Subsequently, Table 9 gives the average computing costs and time complexity on each task of real-word navigation for different components of our ASMAC. The mathematical modeling of total per-batch complexity can be formulated as follows:

$$O_{ASMAC} = O_{LAM} + O_{SMRR} + O_{AM} + O_{PPO} \quad (19)$$

$$= O(M \cdot N_m \cdot (D^2 + h \cdot D) + N + n \cdot P + P \cdot N)$$

where $D = 128$ is the embedding dimension of pre-encoding network, and P is the number of model parameters in the actor-critic network. O_{LAM} is the LAM time complexity.

Our ASMAC system without any code optimization is implemented within ROS. Clearly, it shows that the PPO component has highest time complexity, which requires the most run time, occupying 87.01% among the total computation cost. This mainly because this process contains three time-consuming high-dimensional matrix operations, i.e., objective clip, advantage estimation, and gradient calculation. The computation of SMRR and AM are very efficient, requiring only 1.22s in total. Besides, the LAM cost 1.04s, which can be further accelerated in a parallel computation way. It is concluded by formalizing the time complexity and computing cost that our ASMAC's experimental performance is not just effective but also computationally feasible for real-world deployment.

5. Conclusion

A novel collision avoidance method, ASMAC, is presented in this paper for efficient and safe AGV navigation in complex dynamic environments. AM mechanism is combined with PPO algorithm to form a novel actor-critic RL model. It can boost sample efficiency in navigation policy learning to optimize and accelerate the model training. Then, we extend SMRR and LAM to the actor-critic paradigm, which improve model perception ability to make effective and safe collision-free decision rapidly. Besides, MLP reward setting is devised to predict

Table 9

Average elapsed cost and time complexity of different components on real-word navigation.

Components	Complexity	Time (s)	Ratio (%)
LAM	$O(M \cdot N_m \cdot (D^2 + h \cdot D))$	1.04	5.96
SMRR	$O(N)$	0.36	2.28
AM	$O(n \cdot P)$	0.86	4.75
PPO	$O(P \cdot N)$	15.12	87.01
Total	O_{ASMAC}	17.38	100

the future events of collision threat, which can not only evaluate the impact of current action on future AGV-environment interaction, but also prevent the AGV from generating intrusive, shortsighted, and unnatural motion policies. Finally, a great deal of navigation experiments is built in complex dynamic scenes. The experimental result analysis of different configurations demonstrates the specific effect of different algorithm elements. By comparing our ASMAC with other up-to-date methods, improved safety and generalization, and preserving a higher efficiency than other counterparts, are indicated.

The ASMAC approach presented in this work has realized many satisfactory properties, but it still has some limitations that need further to be researched in the future. On the one hand, the fast obstacle motion interferences are not considered in current navigation system, which may introduce difficulty in extracting hidden state relevant to collision avoidance task. To this end, a local temporal-spatial graph-based social encoder with multi-sensor fusion will be developed and then embedded into ASMAC system. On the other hand, the approach solely focuses on local collision avoidance and does not consider global path planning. This may lead to locally optimal but globally inefficient path, especially in large environments. Hence, we will attempt to combine the system with classical mapping mechanism (e.g., SLAM) to obtain satisfactory global collision-free navigation in a large dynamic workspace.

CRedit authorship contribution statement

Chao Sun: Writing – review & editing, Validation, Supervision, Software, Resources, Funding acquisition, Formal analysis. **Jiang Wang:** Writing – review & editing. **Xing Wu:** Writing – review & editing, Resources, Funding acquisition. **Chaoxu Mu:** Supervision, Conceptualization. **Changyin Sun:** Validation, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is supported in part by the National Natural Science Foundation of China under Grant 52475521, 62495083 and 62503005, and the Anhui Provincial Natural Science Foundation under Grant 2408085QF208.

Data availability

No data was used for the research described in the article.

References

- Adam, K.D.B.J., et al., 2014. A method for stochastic optimization. 1412, (6), arXiv preprint arXiv:1412.6980.
- Bengio, Y., Louradour, J., Collobert, R., Weston, J., 2009. Curriculum learning. In: Proc. Int. Conf. Mach. Learn.. ICML, pp. 41–48.
- Chen, Y., Peng, H., Grizzle, J., 2018. Obstacle avoidance for low-speed autonomous vehicles with barrier function. IEEE Trans. Control Syst. Technol. 26 (1), 194–206.
- Cheng, J., et al., 2022. A review of visual SLAM methods for autonomous driving vehicles. Eng. Appl. Artif. Intell. 114, 104992.
- Dong, L., He, Z., Song, C., Yuan, X., Zhang, H., 2024. Multi-robot social-aware cooperative planning in pedestrian environments using attention-based actor-critic. Artif. Intell. Rev. 57 (4), 108.
- Everett, M., Chen, Y.F., How, J.P., 2021. Collision avoidance in pedestrian-rich environments with deep reinforcement learning. IEEE Access 9, 10357–10377.
- Fan, T., Long, P., Liu, W., Pan, J., 2020. Distributed multi-robot collision avoidance via deep reinforcement learning for navigation in complex scenarios. Int. J. Robot. Res. 39 (7), 856–892.
- Godoy, J., Karamouzas, I., Guy, S.J., Gini, M.L., 2016. Moving in a crowd: Safe and efficient navigation among heterogeneous agents. In: Proc. Int. Joint Conf. Artif. Intell.. IJCAI, pp. 294–300.
- Gu, T., Chen, G., Li, J., Lin, C., Rao, Y., Zhou, J., Lu, J., 2022. Stochastic trajectory prediction via motion indeterminacy diffusion. In: Proc. IEEE/CVF Comput. Vis. Pattern Recognit. pp. 17113–17122.
- Guo, B., Guo, N., Cen, Z., 2022. Obstacle avoidance with dynamic avoidance risk region for mobile robots in dynamic environments. IEEE Robot. Autom. Lett. 7 (3), 5850–5857.
- He, Z., Dong, L., Sun, C., Wang, J., 2022. Asynchronous multithreading reinforcement-learning-based path planning and tracking for unmanned underwater vehicle. IEEE Trans. Syst. Man Cybern. Syst. 52 (5), 2757–2769.
- Huang, Y., Ding, H., Zhang, Y., Wang, H., Cao, D., Xu, N., Hu, C., 2019. A motion planning and tracking framework for autonomous vehicles based on artificial potential field elaborated resistance network approach. IEEE Trans. Ind. Electron. 67 (2), 1376–1386.
- Kiumarsi, B., Vamvoudakis, K.G., Modares, H., Lewis, F.L., 2017. Optimal and autonomous control using reinforcement learning: A survey. IEEE Trans. Neural Netw. Learn. Syst. 29 (6), 2042–2062.
- Koenig, N., Howard, A., 2004. Design and use paradigms for gazebo, an open-source multi-robot simulator. IROS, In: Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst., vol. 3, pp. 2149–2154.
- Lee, D.H., Lee, S.S., Ahn, C.K., Shi, P., Lim, C.-C., 2020. Finite distribution estimation-based dynamic window approach to reliable obstacle avoidance of mobile robot. IEEE Trans. Ind. Electron. 68 (10), 9998–10006.
- Lin, C., Cheng, Y., Wang, X., Yuan, J., Wang, G., 2023. Transformer-based dual-channel self-attention for UAV autonomous collision avoidance. IEEE Trans. Intell. Veh. 8 (3), 2319–2331.
- Lin, H.-I., Shodiq, M.A.F., Hsieh, M.F., 2025. Robot path planning based on three-dimensional artificial potential field. Eng. Appl. Artif. Intell. 144, 110127.
- Liu, S., et al., 2023. Intention aware robot crowd navigation with attention-based interaction graph. In: Proc. IEEE Int. Conf. Robot. Automat.. ICRA, pp. 12015–12021.
- Long, P., Fan, T., Liao, X., Liu, W., Zhang, H., Pan, J., 2018. Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning. In: Proc. IEEE Int. Conf. Robot. Automat.. ICRA, pp. 6252–6259.
- Malone, N., Chiang, H.-T., Lesser, K., Oishi, M., Tapia, L., 2017. Hybrid dynamic moving obstacle avoidance using a stochastic reachable set-based potential field. IEEE Trans. Robot. 33 (5), 1124–1138.
- Missura, M., Bennewitz, M., 2019. Predictive collision avoidance for the dynamic window approach. In: Proc. IEEE Int. Conf. Robot. Automat.. ICRA, pp. 8620–8626.
- Mnih, V., et al., 2015. Human-level control through deep reinforcement learning. Nature 518, 529–533.
- Mnih, V., et al., 2016. Asynchronous methods for deep reinforcement learning. In: Proc. Int. Conf. Mach. Learn.. pp. 1928–1937.
- Oliehoek, F.A., Amato, C., et al., 2016. A Concise Introduction to Decentralized POMDPs, vol. 1, Springer.
- Pham, H., Pham, Q.-C., 2018. A new approach to time-optimal path parameterization based on reachability analysis. IEEE Trans. Robot. 34 (3), 645–659.
- Poonganam, S.N.J., Gopalakrishnan, B., Avula, V.S.S.B.K., Singh, A.K., Krishna, K.M., Manocha, D., 2020. Reactive navigation under non-parametric uncertainty through hilbert space embedding of probabilistic velocity obstacles. IEEE Robot. Autom. Lett. 5 (2), 2690–2697.
- Qureshi, A.H., Miao, Y., Simeonov, A., Yip, M.C., 2021. Motion planning networks: Bridging the gap between learning-based and classical motion planners. IEEE Trans. Robot. 37 (1), 48–66.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P., 2018. High-dimensional continuous control using generalized advantage estimation. arXiv preprint arXiv:1506.02438.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
- Seo, H., Lee, D., Son, C.Y., Tomlin, C.J., Kim, H.J., 2019. Robust trajectory planning for a multirotor against disturbance based on hamilton-jacobi reachability analysis. In: Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.. IROS, pp. 3150–3157.
- Seo, H., Son, C.Y., Lee, D., Kim, H.J., 2020. Trajectory planning with safety guaranty for a multirotor based on the forward and backward reachability analysis. In: Proc. IEEE Int. Conf. Robot. Automat.. ICRA, pp. 7142–7148.
- Shen, S., Mulgaonkar, Y., Michael, N., Kumar, V., 2014. Multi-sensor fusion for robust autonomous flight in indoor and outdoor environments with a rotorcraft mav. In: Proc. IEEE Int. Conf. Robot. Automat.. ICRA, pp. 4974–4981.
- Sheng, C., Hao, A., Jun, N., Haixia, W., Xiao, L., 2025. Improved M-DQN with ϵ -UCB action selection policy and multi-goal fusion reward function for mobile robot path planning. IEEE Trans. Veh. Technol. 74 (4), 5358–5370. <http://dx.doi.org/10.1109/TVT.2024.3503552>.
- Soares, E.F.d.S., Campos, C.A.V., 2025. Self-supervised learning of monocular depth estimators in autonomous vehicles with federated learning. Eng. Appl. Artif. Intell. 151, 110572.
- Song, C., He, Z., Dong, L., 2024. A local-and-global attention reinforcement learning algorithm for multiagent cooperative navigation. IEEE Trans. Neural Netw. Learn. Syst. 35 (6), 7767–7777.
- Sui, Z., Pu, Z., Yi, J., Wu, S., 2020. Formation control with collision avoidance through deep reinforcement learning using model-guided demonstration. IEEE Trans. Neural Netw. Learn. Syst. 32 (6), 2358–2372.
- Sun, C., Wu, X., Sun, J., Sun, C., Dong, L., 2022. Robust pose estimation via hybrid point and twin line reprojection for rgb-d vision navigation. IEEE Trans. Instrum. Meas. 71, 1–19.
- Sun, C., Wu, X., Sun, J., Sun, C., Xu, M., Ge, Q., 2023. Saliency-induced moving object detection for robust RGB-d vision navigation under complex dynamic environments. IEEE Trans. Intell. Transp. Syst. 24 (10), 10716–10734.
- Sun, C., Wu, X., Wang, Y., Sun, C., 2024. Attention-based value classification reinforcement learning for collision-free robot navigation. IEEE Trans. Intell. Veh. 9 (11), 6898–6911.
- Wu, X., Sun, C., Zou, T., Li, L., Wang, L., Liu, H., 2020. Svm-based image partitioning for vision recognition of agv guide paths under complex illumination conditions. Robot. Comput.-Integr. Manuf. 61, 1–14.
- Wu, Q., Xu, K., Wang, J., Xu, M., Gong, X., Manocha, D., 2021. Reinforcement learning-based visual navigation with information-theoretic regularization. IEEE Robot. Autom. Lett. 6 (2), 731–738.
- Xia, J., Zhu, X., Liu, Z., Luo, Y., Wu, Z., Wu, Q., 2023. Research on collision avoidance algorithm of unmanned surface vehicle based on deep reinforcement learning. IEEE Sensors J. 23 (11), 11262–11273.
- Xiao, H., Li, Z., Chen, C.P., 2016. Formation control of leader-follower mobile robots' systems using model predictive control based on neural-dynamic optimization. IEEE Trans. Ind. Electron. 63 (9), 5752–5762.
- Xu, M., Wang, S., Hu, Y., Tang, H., Cong, R., Nie, L., 2025. Cross-model nested fusion network for salient object detection in optical remote sensing images. IEEE Trans. Cybern. 55 (11), 5332–5345.
- Zhang, B., et al., 2024. A reliable traversability learning method based on human-demonstrated risk cost mapping for mobile robots over uneven terrain. Eng. Appl. Artif. Intell. 138, 109339.
- Zhou, C., Huang, B., Fränti, P., 2024. Representation learning and reinforcement learning for dynamic complex motion planning system. IEEE Trans. Neural Netw. Learn. Syst. 35 (8), 11049–11063.



Chao Sun received the M.S. degree in mechanical and electrical engineering from the College of Mechanical and Electrical Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2020, and the Ph.D. degree in control science and engineering from the College of Electronics and Information Engineering, Tongji University, Shanghai, China, in 2024. He is currently a Lecturer with the School of Artificial Intelligence, Anhui University, Hefei, China. He has published more than 10 international articles in ranked journal. His research interests include deep reinforcement learning, computer vision, and robotics.



Jiang Wang received the B.E. degree in computer science and technology from the College of Information Engineering, Henan University of Science and Technology, Luoyang, China, in 2023. He is currently pursuing the M.E. degree with the School of Artificial Intelligence, Anhui University, Hefei, China. His research interests include machine learning, deep learning, reinforcement learning, robot navigation, and path planning.



Xing Wu received the B.Eng. and Ph.D. degrees in Mechanical Engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2004 and 2010, respectively. In 2010, he joined the College of Mechanical and Electrical Engineering, NUAA, where he is currently an associate professor. He was a visiting professor at the Centre for Intelligent Machines, McGill University, Canada, from Nov. 2016 to Nov. 2017. His research interests include: intelligent sensing and control; design, navigation and control of mobile robots; and coordinated control and intelligent scheduling of multi-robot systems.



Chaouxu Mu received the Ph.D. degree in control science and engineering from the School of Automation, Southeast University, Nanjing, China, in 2012. She was a Visiting Ph.D. Student with the Royal Melbourne Institute of Technology University, Melbourne, VIC, Australia, from 2010 to 2011. She was a Post-Doctoral Fellow with the Department of Electrical, Computer and Biomedical Engineering, The University of Rhode Island, Kingston, RI, USA, from 2014 to 2016. She is currently a Professor with the School of Electrical and Information Engineering, Tianjin University, Tianjin, China. She has authored more than 100 journals and conference papers, and co-authored two monographs. Her current research interests include nonlinear system control and optimization and adaptive and learning systems.



Changyin Sun received his B.S. degree from the Department of Mathematics, Sichuan University, Chengdu, in 1996, and M.S. and Ph.D. degrees in electrical engineering from Southeast University, Nanjing, China, in 2001 and 2003, respectively. He was a Post Doctoral Fellow with the Department of Computer Science of Hong Kong in 2004. Since 2007, he has been a Professor at the School of Automation, Southeast University. From 2022, he served as a member of the Standing Committee of the Party Committee and Vice President of Anhui University, Dean of the Future College, and Dean of the School of Artificial Intelligence. He served as a CAAI Fellow in 2023. He is the Associate Editor of IEEE Transactions on Neural Networks and Learning Systems, IEEE Neural Processing Letters, and IEEE/CAA Journal of Automatica Sinica. His current research interests include intelligent control, pattern recognition, machine vision, theory and design of intelligent control systems, and deep reinforcement learning.